DualHash: A Stochastic Primal-Dual Algorithm with Theoretical Guarantee for Deep Hashing

Luxuan Li*, Xiao Wang†, and Chunfeng Cui‡

Abstract. Deep hashing encodes high-dimensional image features into compact binary codes, enabling efficient similarity search in large-scale content-based image retrieval, a problem akin to semantic-preserving lossy compression under extreme bit-rate constraints. A fundamental challenge in deep hashing stems from the discrete nature of quantization in generating the codes. W-type regularizations, such as $||z| - 1|$, have been proven effective as they encourage variables toward binary values. However, existing methods often directly optimize these regularizations without convergence guarantees. While proximal gradient methods offer a promising solution, the coupling between W-type regularizers and neural network outputs results in composite forms that generally lack closed-form proximal solutions. In this paper, we present a stochastic primal-dual hashing algorithm, referred to as DualHash, that provides rigorous complexity bounds. Using Fenchel duality, we partially transform the nonconvex W-type regularization optimization into the dual space, which results in a proximal operator that admits closed-form solutions. We derive two algorithm instances: a momentum-accelerated version with $\mathcal{O}(\varepsilon^{-4})$ complexity and an improved $\mathcal{O}(\varepsilon^{-3})$ version using variance reduction. Experiments on three standard datasets demonstrate superior retrieval accuracy and reduced quantization error, validating the practical value of DualHash for image retrieval problems. Our implementation of DualHash is available at https://github.com/RailLee/DualHash.

Key words. nonconvex nonsmooth optimization, primal-dual method, deep hashing, image retrieval, stochastic optimization, oracle complexity

MSC codes. 90C26, 90C46, 65K05, 68T07, 68P20

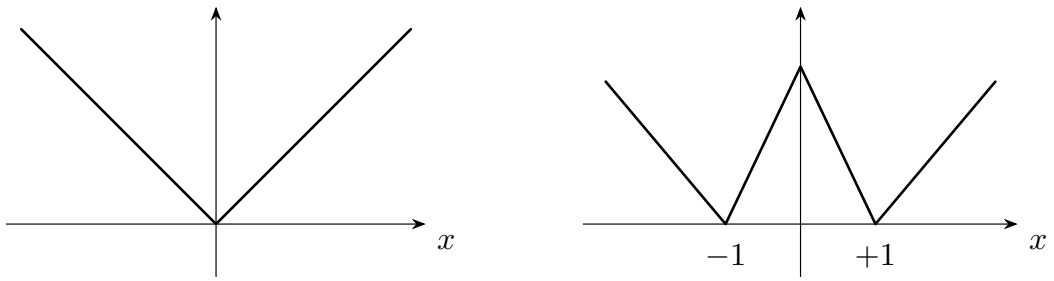
1. Introduction.

Hashing is a pivotal technique in large-scale image retrieval, which encodes image content into compact binary codes (e.g., ± 1) for efficient storage and accurate similarity search. Mathematically, this amounts to a *constrained projection* onto the binary hypercube $\{-1, +1\}^d$ that preserves semantic neighborhood structures, a nonconvex discrete optimization problem akin to lossy image compression, where the fidelity criterion is pairwise similarity rather than pixel-level reconstruction. Deep hashing leverages neural networks to learn such projections, achieving compression ratios up to $512\times$ while maintaining competitive retrieval accuracy [41]. The mainstream approaches fall into two categories: sign-based methods and continuous relaxation. A fundamental challenge, however, is that the sgn function has zero gradient almost everywhere, rendering standard first-order methods ineffective [8]. Continuous relaxation mitigates this but inevitably introduces *quantization error*—the discrepancy between optimized continuous values and the final binary codes [60]. To mitigate this issue, researchers have developed specialized regularization techniques [7, 36, 39]. Among them, W-type regularizations, such as $||\boldsymbol{z}| - 1|$, have proven effective. These regularizers impose sharp penalties to encourage variables towards binary values (e.g., -1 or 1). Mathematically, this approach naturally

*School of Mathematical Sciences, Beihang University, Beijing, 100191, China (luxuanli@buaa.edu.cn).

†School of Computer Science and Engineering, Sun Yat-Sen University, Guangzhou, 510006, China (wangx936@mail.sysu.edu.cn).

‡School of Mathematical Sciences, Beihang University, Beijing, 100191, China (chunfengcui@buaa.edu.cn).



(a) $h(\boldsymbol{z}) = \|\boldsymbol{z}\|_1$ (b) $h(\boldsymbol{z}) = \||\boldsymbol{z}| - 1\|_1$

Figure 1.1: The V-type (left) versus W-type (right) regularizations.

formulates deep hashing as a nonconvex composite optimization problem[1]:

$$\min_{\boldsymbol{x}\in\mathbb{R}^{d_{\boldsymbol{x}}}} \quad \frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{x}) + \sum_{i=1}^{n} h(\boldsymbol{D}_i(\boldsymbol{x})), \tag{1.1}$$

where f_i and $\boldsymbol{D}_i$ are continuously differentiable and h is a proper and lower semi-continuous (l.s.c.) regularizer. Classic examples include the ℓ_1 norm to introduce sparsity and the ℓ_2 norm weight decay to prevent overfitting. In practice, researchers typically apply stochastic (sub)gradient methods (SGD) directly to optimize the objective (1.1), and deep hashing follows this paradigm as well. This seemingly effective choice, however, frequently leads to suboptimal performance. The regularizer is often nonsmooth around some regions; using subgradients may result in slow convergence and oscillation.

The proximal operator is a powerful tool for handling nonsmooth regularizers, with widespread applications in signal/image processing and computer vision, including sparse recovery [64], inverse imaging problems [19, 50, 35], low-rank and robust matrix factorization [69, 12], and structured neural network training [4, 31]. For the special case $\boldsymbol{D}_i(\boldsymbol{x}) = \boldsymbol{x}$, stochastic proximal gradient methods (SPGD) and their variance-reduced or momentum-based variants have been extensively studied [38, 62]. However, these methods require regularizers with tractable proximal operators (e.g., ℓ_1 soft-thresholding). Although the W-type regularizer admits a closed-form proximal operator [2] (Figure 1.1), its composition with neural network outputs $h(\boldsymbol{D}_i(\boldsymbol{x}))$ renders the proximal subproblem intractable, making SPGD inapplicable to Problem (1.1). Moreover, unlike standard deep learning tasks, where regularization is typically imposed on model parameters, deep hashing requires regularization directly on the network output to enforce binary codes, as illustrated in Figure 1.2. This makes the nonsmooth composite form $h(\boldsymbol{D}_i(\boldsymbol{x}))$ intrinsic to deep hashing and therefore necessary to handle explicitly.

For nonsmooth compositional optimization, including Problem (1.1), significant research has been conducted when $h \circ \boldsymbol{D}$ is either convex [68, 54] or weakly convex [17, 21, 56, 29]. For instance, $h \circ \boldsymbol{D}$ is weakly convex when h is convex and $\boldsymbol{D}$ is smooth [17, 21, 56], or when h is component-wise nondecreasing and $\boldsymbol{D}$ is weakly convex [29]. However, when h itself is nonconvex, as in our setting, $h \circ \boldsymbol{D}$ is not necessarily weakly convex, rendering these methods inapplicable. For the case where h is nonconvex, [5] reformulates the problem as a constrained

[1]A formal definition of this problem is provided in Section 3.

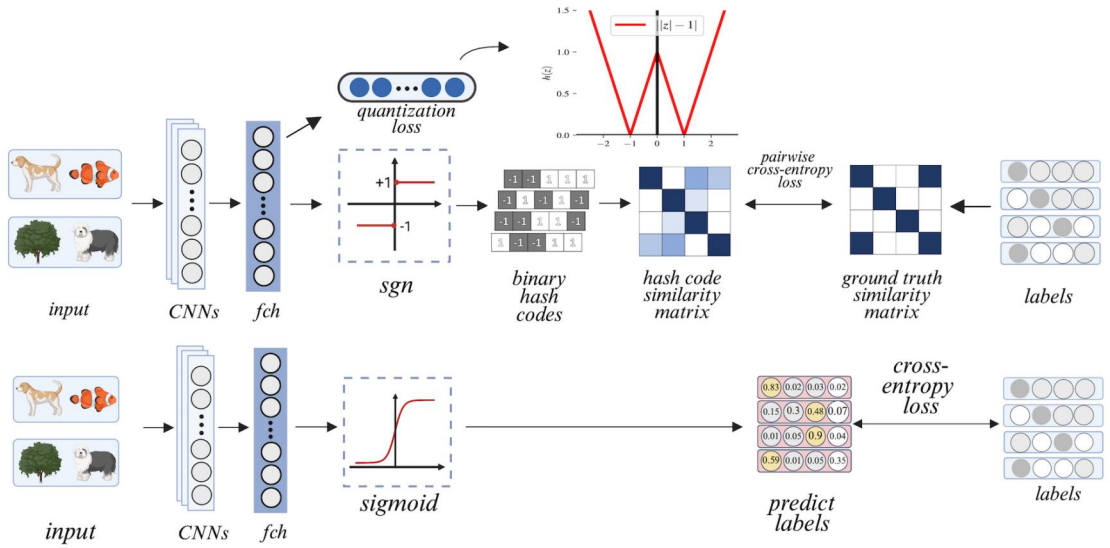


Figure 1.2: Comparison between deep learning for hashing and classification.

optimization by imposing redundant variables and establishes asymptotic convergence via a Lagrangian-based approach. When h is Lipschitz continuous but possibly nonconvex, Yao et al.[63] establish an $\mathcal{O}(1/(\delta\epsilon^2))$ iteration complexity for finding a (δ, ϵ)-stationary point. More recently, Zhang et al.[66] provide a systematic analysis for the cases where h is nonconvex with or without Lipschitz continuity. Among their results most relevant to ours, they establish an $\mathcal{O}(\epsilon_1^{-2}\epsilon_2^{-1})$ iteration complexity for finding an $(\epsilon_1, \epsilon_2, 0)$-stationary point for Lipschitz continuous h, along with subsequential convergence results. However, all consider only the *deterministic* setting, whereas ours addresses the *stochastic* finite-sum formulation that naturally arises in large-scale deep hashing; directly extending their results to our setting is nontrivial.

1.1. Contributions. In this paper, we propose an efficient deep hashing algorithm to address these challenges posed by nonlinear composition with W-type regularization. Our approach reformulates the W-type regularized problem using a decoupling scheme and develops a new primal-dual algorithm based on Fenchel duality. We aim to bridge the theoretical-practical gap by providing rigorous non-asymptotic convergence analysis and enhanced practical performance in deep hashing. Our main contributions are as follows:

- **Novel primal-dual hashing algorithms.** We reformulate deep hashing models with W-type regularization as a finite-sum muti-block optimization problem (P), decoupling the challenging composite term $h(\boldsymbol{D}(\boldsymbol{x}))$ and facilitating sequential convergence analysis. Grounded in this formulation, we propose *DualHash*, a stochastic primal-dual algorithm for deep hashing. Using Fenchel duality, we partially transform the nonconvex W-type regularization optimization into a convex dual formulation that admits closed-form updates, ensuring robustness and stable convergence. We then develop two important instances: a momentum-based implementation (**DualHash-StoM**) and a variance-reduced implementation using StoRM (**DualHash-StoRM**).

Table 1.1: Comparison of optimization and convergence guarantee of deep hashing methods.

Type	Method	Model	Quantization Loss	Optimization	Complexity
Discrete	DDSH [30]	$\min L_s(\boldsymbol{b}) + Q(\boldsymbol{u}, \boldsymbol{b})$	$\|\cdot\|^2$	BQP	
	DSDH [36] DPSH [37]	$\min L_s(\boldsymbol{u}) + Q(\boldsymbol{u}, \boldsymbol{b})$	$\|\cdot\|^2$	*sgn* update DCC	
Continuous	DCH [7]	$\min L_s(\boldsymbol{u}) + h(\boldsymbol{u})$	$\log\left(1 + \frac{\mathrm{dist_h}(\lvert\boldsymbol{u}\rvert, \mathbf{1})}{\gamma}\right)$	SGDM	—
	DNNH [33]		$\mathcal{I}_{[-1,1]^d}(\boldsymbol{u})$		
	DSH [39]		$\|\|\boldsymbol{u}\| - \mathbf{1}\|_1$		
	DTSH [57]		$\|\|\boldsymbol{u}\| - \mathbf{1}\|_2^2$		
	DHN [71]		$\sum_i \log(\cosh(\lvert u_i\rvert - 1))$		
	MDSHC [55]	$\min L_c(\boldsymbol{u}) + L_s(\boldsymbol{u}) + h(\boldsymbol{u})$	$\|\|\boldsymbol{u}\| - \mathbf{1}\|_1$	RMSProp	
	HashNet [8]	$\min L_s(\boldsymbol{u})$	-	SGDM	
	OrthoHash [28]			Adam	
	DualHash-StoM	$\min L_s(\boldsymbol{u}) + Q(\boldsymbol{u}, \boldsymbol{b}) + h(\boldsymbol{b})$	$\|\|\boldsymbol{b}\| - \mathbf{1}\|_1$	Sto Primal-Dual	$\mathcal{O}(\varepsilon^{-4})$
	DualHash-StoRM				$\mathcal{O}(\varepsilon^{-3})$

Notes: The **Model** column shows the core optimization objective, where $\boldsymbol{b}$ denotes binary codes and $\boldsymbol{u}$ denotes continuous outputs. The **Quantization Loss** column specifies the loss ($Q(\cdot,\cdot)$ or $h(\cdot)$) for quantization error, with '-' indicating no explicit loss. DCC stands for Discrete Cyclic Coordinate, while BQP denotes Binary Quadratic Programming. The **Complexity** column reports the convergence rate, with '-' indicating no such analysis.

- **Comparable complexity guarantee.** We provide rigorous non-asymptotic complexity analysis for deep hashing optimization under the challenging stochastic, nonconvex, multi-block primal-dual framework. Specifically, to attain stochastic critical points of Problem (P), we establish an $\mathcal{O}(\varepsilon^{-4})$ oracle complexity result for DualHash-StoM and, under the mean-square smoothness condition, an improved $\mathcal{O}(\varepsilon^{-3})$ complexity result for DualHash-StoRM, matching the best-known lower bound for stochastic smooth optimization [1]. Table 1.1 summarizes the comparison with existing deep hashing methods.
- **Superior empirical performance.** Extensive experiments on three widely-used datasets demonstrate the practical effectiveness of DualHash. In particular, the proposed method achieves consistently lower quantization errors across different bit lengths compared to baselines, validating the advantages of our primal-dual framework for image retrieval in both single-label and multi-label settings.

1.2. Related Work.

Optimization in Deep Hashing. W-type regularizers such as $\|\|\boldsymbol{z}| - \mathbf{1}\|_1$ have been widely adopted in deep hashing to bridge the gap between continuous network outputs and discrete binary codes [39, 55, 71]. Alternative strategies either introduce auxiliary hash codes with a smoother surrogate $\|\cdot\|_2^2$ [30, 36, 37], or refine the discretization objective via greedy coding, central similarity targets, or proxy-based supervision [51, 65, 11]. Nevertheless, even recent architectures continue to rely on W-type regularization to mitigate quantization error. For instance, He and Wei [27] employ $\|\|\mathbf{z}| - \mathbf{1}\|_2^2$ as a smooth surrogate for $\|\|\mathbf{z}| - \mathbf{1}\|_1$ to ease gradient-based optimization, highlighting that the core difficulty of developing theoretically grounded methods for nonsmooth W-type regularization under nonlinear network mappings has not been resolved. This is precisely the gap this work aims to fill.

Primal-Dual and Stochastic Splitting Methods. Primal-dual methods, such as PDHG [10] and its unified extensions [22], are foundational tools for composite optimization with broad applications in imaging sciences. These methods enjoy ergodic $\mathcal{O}(1/k)$ convergence and remain stable under adjoint mismatch [13]. For convex nonlinear saddle-point models, Gao and Zhang [24] proposed an alternative extrapolation scheme with provable convergence guarantees. Stochastic counterparts, notably SPDHG [9] and the stochastic majorize-minimize subspace method [14], handle structured finite-sum objectives with almost-sure and ergodic convergence. In the nonconvex setting, Clason et al. [15] established global convergence under the Kurdyka–Łojasiewicz (KL) condition, while Guo et al. [26] further addressed nonconvex stochastic composite problems. More recently, Shumaylov et al. [50] incorporated weakly convex neural-network regularizers into a primal-dual framework and proved convergence of PDHGM to critical points under the KL condition. When the problem admits a block-separable structure, splitting methods such as ADMM with its stochastic variants [58, 70, 12, 3] and stochastic alternative minimization [20, 45, 4] provide promising approaches for nonconvex multi-block optimization.

Stochastic Nonconvex Constrained Optimization. From a constrained optimization perspective, Problem (1.1) can be reformulated as a nonlinearly constrained optimization problem of the form

$$\min_{\boldsymbol{x}\in\mathbb{R}^q} f(\boldsymbol{x}) + h(\boldsymbol{x}) \quad \text{s.t.} \quad \boldsymbol{c}(\boldsymbol{x}) = 0.$$

Several recent works have established oracle complexity results for first-order stochastic augmented-Lagrangian and penalty methods for problems of this type. For instance, Jin and Wang [32] proposed a single-loop stochastic primal-dual method based on a linearized augmented-Lagrangian function, achieving oracle complexities of $\mathcal{O}(\varepsilon^{-6})$ from an arbitrary initial point and $\mathcal{O}(\varepsilon^{-5})$ from a feasible one for locating an ε-KKT point. By incorporating the recursive momentum estimator of [16], Shi et al. [49] developed a momentum-based linearized augmented-Lagrangian method, improving these bounds to $\mathcal{O}(\varepsilon^{-4})$, or the nearly optimal $\mathcal{O}(\varepsilon^{-3})$ when initialized at a nearly feasible point. To certify a stronger notion of stationarity that deterministically enforces constraint violation of at most ε, Lu et al. [40] introduced a linearized quadratic-penalty method with truncated momentum. Under a regularity condition on the constraint gradients, recursive momentum yields the nearly optimal complexity $\widetilde{\mathcal{O}}(\varepsilon^{-3})$.

Notations. We use a, $\boldsymbol{a}$, and $\boldsymbol{A}$ to denote scalars, vectors, and matrices, respectively. Throughout this paper, $\mathbf{1}$ represents a vector of all ones, and i, j serve as indices. For matrix variables $\boldsymbol{B}$, $\boldsymbol{Y}$, and $\boldsymbol{\Lambda}$, we work directly in $\mathbb{R}^{n\times d}$, equipped with the Frobenius inner product $\langle\cdot,\cdot\rangle$ and induced Frobenius norm $\|\cdot\| := \sqrt{\langle\cdot,\cdot\rangle}$. Unless specified otherwise, $\|\cdot\|$ denotes the Euclidean norm for vectors and the Frobenius norm for matrices. $|\cdot|$ denotes element-wise absolute value. For a closed set $\mathcal{C} \subseteq \mathbb{R}^q$, the distance from a point $\boldsymbol{x} \in \mathbb{R}^q$ to $\mathcal{C}$ is defined as $\operatorname{dist}(\boldsymbol{x}, \mathcal{C}) := \min_{\boldsymbol{y}\in\mathcal{C}} \|\boldsymbol{x} - \boldsymbol{y}\|$.

2. Preliminary.

The extended real-valued function $f : \mathbb{R}^q \to (-\infty, +\infty]$ is called *proper* if its domain $\operatorname{dom}(f) := \{\boldsymbol{x} \in \mathbb{R}^q \mid f(\boldsymbol{x}) < +\infty\}$ is nonempty and $\inf_{\boldsymbol{x}\in\mathbb{R}^q} f > -\infty$; it is called *closed* if its epigraph $\operatorname{epi}(f) := \{(\boldsymbol{x}, t) \in \mathbb{R}^{q+1} \mid f(\boldsymbol{x}) \le t\}$ is closed. For a proper, closed, and

convex function f, the *proximal mapping* of f, denoted by $\text{prox}_{\tau f}(\boldsymbol{y})$, is defined as

$$\text{prox}_{\tau f}(\boldsymbol{y}) := \underset{\boldsymbol{x}\in\text{dom}(f)}{\text{argmin}} \left\{ f(\boldsymbol{x}) + \frac{1}{2\tau}\|\boldsymbol{x}-\boldsymbol{y}\|^2 \right\},$$

where τ is a positive scalar parameter. The proximal mapping admits the following properties.

Proposition 2.1. [47] *Let $f : \mathbb{R}^q \to \mathbb{R}\cup\{+\infty\}$ be proper and closed. Then, the following holds.*

(i) The set $\text{prox}_{\tau f}(\boldsymbol{x})$ is nonempty and compact for any $\boldsymbol{x}\in\mathbb{R}^q$ and $\tau > 0$.

(ii) If f is convex, then $\text{prox}_{\tau f}(\boldsymbol{x})$ contains exactly one value for any $\boldsymbol{x}\in\mathbb{R}^q$ and $\tau > 0$.

We use the StoRM (stochastic recursive momentum) variance-reduction estimator [16]. In stochastic optimization problems where $F(\boldsymbol{x}) = \mathbb{E}_\xi[F(\boldsymbol{x};\xi)]$ with $\xi\in\Xi$, standard momentum methods use

$$\boldsymbol{d}^k = \rho_k \nabla F(\boldsymbol{x}^k,\xi^k) + (1-\rho_k)\boldsymbol{d}^{k-1}, \text{ where } \rho_k\in[0,1],$$

yielding a biased estimate of $\nabla F(\boldsymbol{x}^k)$. StoRM adds the correction term $(1-\rho_{k-1})(\nabla F(\boldsymbol{x}^k,\xi^k) - \nabla F(\boldsymbol{x}^{k-1},\xi^k))$, resulting in:

$$\boldsymbol{d}^k = \begin{cases} \nabla F\left(\boldsymbol{x}^k,\xi^k\right), & \text{if } k=1,\\ \nabla F\left(\boldsymbol{x}^k,\xi^k\right) + (1-\rho_{k-1})\left(\boldsymbol{d}^{k-1} - \nabla F\left(\boldsymbol{x}^{k-1},\xi^k\right)\right), & \text{if } k>1, \end{cases}$$

This estimator exploits the smoothness of $F(\cdot,\xi)$, leading to efficient variance reduction. When $\rho_k = 1$, $\boldsymbol{d}^k$ reduces to vanilla mini-batch SGD. We set $\rho_k\in(0,1)$ for $k\geq 1$ in this paper.

A function $f:\mathbb{R}^q\to\mathbb{R}$ is said to be L_f-smooth if $\|\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{x}')\| \leq L_f\|\boldsymbol{x}-\boldsymbol{x}'\|$ for all $\boldsymbol{x}$, $\boldsymbol{x}'\in\mathbb{R}^q$. A mapping $\mathcal{M}:\mathcal{D}\to\mathbb{R}^l$ is said to be C-Lipschitz continuous if $\|\mathcal{M}(\boldsymbol{x}) - \mathcal{M}(\boldsymbol{x}')\| \leq C\|\boldsymbol{x}-\boldsymbol{x}'\|$ for all $\boldsymbol{x},\boldsymbol{x}'\in\mathcal{D}$. For an L_f-smooth function, the descent lemma states that

$$f(\boldsymbol{x}) \leq f(\boldsymbol{x}') + \langle\nabla f(\boldsymbol{x}'), \boldsymbol{x}-\boldsymbol{x}'\rangle + \frac{L_f}{2}\|\boldsymbol{x}-\boldsymbol{x}'\|^2.$$

3. Problem Reformulation.

3.1. W-type Regularized Deep Hashing Model.

Deep hashing aims to learn network parameters $\boldsymbol{x}\in\mathbb{R}^{d_{\boldsymbol{x}}}$ that generate binary codes preserving semantic similarity in the Hamming space. A direct formulation is

$$\min_{\boldsymbol{x}\mathbb{R}^{d_{\boldsymbol{x}}}} \frac{1}{n}\sum_{i=1}^{n} \ell_{s,i}\big(\text{sgn}(\boldsymbol{C}_i(\boldsymbol{x}))\big),$$

where n is the number of training samples, $\boldsymbol{C}_i(\boldsymbol{x})\in\mathbb{R}^d$ is the output of the i-th sample, and $\ell_{s,i}$ is a similarity-preserving loss (e.g., pairwise cross-entropy [71]). This objective is hard to optimize by backpropagation because $\text{sgn}(\cdot)$ is nondifferentiable and has zero gradient almost everywhere.

To address this issue, W-type regularization (e.g., $h(z) = \lambda\big||z|-1\big|$) is introduced to drive continuous outputs toward $\{-1,1\}$. Specifically, the model is trained to produce the i-th

continuous network output $\boldsymbol{D}_i(\boldsymbol{x})$ with W-type regularization h, and binary codes are obtained post-training via $\boldsymbol{b}_i = \operatorname{sgn}(\boldsymbol{D}_i(\boldsymbol{x}))$. Consequently, we formulate the core optimization problem for W-type regularized deep hashing methods as follows

$$\min_{\boldsymbol{x}\in\mathbb{R}^{d_{\boldsymbol{x}}}} \quad \frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{x}) + \sum_{i=1}^{n} h(\boldsymbol{D}_i(\boldsymbol{x})), \tag{3.1}$$

where $f_i(\boldsymbol{x}) = \ell_{s,i}(\boldsymbol{x}) + \frac{\mu}{2}\|\boldsymbol{x}\|^2$ with $\mu > 0$ to ensure coercivity.

3.2. Structured Reformulation. We then employ variable splitting by introducing auxiliary variables $\boldsymbol{b}_i = \boldsymbol{D}_i(\boldsymbol{x})$ for all $i = 1, \ldots, n$, and then apply the quadratic penalty method to obtain the following two-block finite-sum optimization problem:

$$\text{(P)} \qquad \min_{\boldsymbol{x}\in\mathbb{R}^{d_{\boldsymbol{x}}},\boldsymbol{B}\in\mathbb{R}^{n\times d}} J(\boldsymbol{x},\boldsymbol{B}) = \overbrace{\underbrace{\frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{x})}_{f(\boldsymbol{x})} + \underbrace{\frac{\gamma}{2n}\sum_{i=1}^{n}\|\boldsymbol{D}_i(\boldsymbol{x}) - \boldsymbol{b}_i\|^2}_{P(\boldsymbol{x},\boldsymbol{B})}}^{F(\boldsymbol{x},\boldsymbol{B})=\frac{1}{n}\sum_{i=1}^{n} F_i(\boldsymbol{x},\boldsymbol{b}_i)} + \underbrace{\sum_{i=1}^{n} h(\boldsymbol{b}_i)}_{h(\boldsymbol{B})}.$$

where we define $\boldsymbol{B} = [\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_n]$ and $\gamma > 0$ as the penalty parameter. This reformulation decomposes the original problem (3.1) into simpler structural blocks, enabling us to exploit the problem structure better. Building upon Problem (P), we develop a stochastic primal-dual algorithm that addresses the lack of tractable proximal mappings in the nonsmooth composite form and the absence of convergence guarantees in existing deep hashing methods.

4. Stochastic Primal-Dual Hashing Algorithms.

4.1. Algorithmic Framework. Let us consider the equivalent constrained formulation of Problem (P):

$$\begin{aligned} \min_{\boldsymbol{x}\in\mathbb{R}^{d_{\boldsymbol{x}}},\boldsymbol{B}\in\mathbb{R}^{n\times d},\boldsymbol{Y}\in\mathbb{R}^{n\times d}} \quad & F(\boldsymbol{x},\boldsymbol{B}) + h(\boldsymbol{Y}) \\ \text{s.t.} \quad & \boldsymbol{Y} = \boldsymbol{B}, \end{aligned}$$

whose corresponding dual problem is

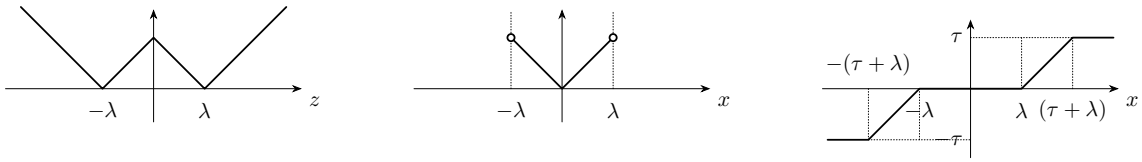
$$\text{(D)} \qquad \max_{\boldsymbol{\Lambda}\in\mathbb{R}^{n\times d}} \min_{\boldsymbol{x}\in\mathbb{R}^{d_{\boldsymbol{x}}},\boldsymbol{B}\in\mathbb{R}^{n\times d},\boldsymbol{Y}\in\mathbb{R}^{n\times d}} F(\boldsymbol{x},\boldsymbol{B}) + h(\boldsymbol{Y}) + \langle\boldsymbol{\Lambda}, \boldsymbol{B} - \boldsymbol{Y}\rangle.$$

Recalling the definition of the conjugate function [47]:

$$h^*(\boldsymbol{x}) := \sup_{\boldsymbol{y}\in\mathbb{R}^n}\{\langle\boldsymbol{x},\boldsymbol{y}\rangle - h(\boldsymbol{y})\}, \quad \boldsymbol{x}\in\mathbb{R}^n \tag{4.1}$$

and observing that the minimization with respect to $\boldsymbol{Y}$ in Problem (D) can be performed independently, we then simplify the Lagrangian function in the problem (D) to

$$\mathcal{L}(\boldsymbol{x},\boldsymbol{B},\boldsymbol{\Lambda}) := F(\boldsymbol{x},\boldsymbol{B}) + \langle\boldsymbol{B},\boldsymbol{\Lambda}\rangle - h^*(\boldsymbol{\Lambda}). \tag{4.2}$$



(a) $h(z) = \lambda \||z| - 1\|_1$, $\lambda = 1$ (b) $h^*(x)$, $\lambda = 1$ (c) $\text{prox}_{h^*}(x)$, $\lambda = \tau = 1$

Figure 4.1: A W-type function, its conjugate function, and the proximal mapping of h^*.

Through this transformation, we replace $h(\boldsymbol{Y})$ with $-h^*(\boldsymbol{\Lambda})$ and optimize the h^* in the dual space. The convexity of h^* makes $\text{prox}_{h^*}(\cdot)$ often much easier to compute (potentially having a closed-form solution) than $\text{prox}_h(\cdot)$. For the W-type regularization on the matrix variable $\boldsymbol{B} \in \mathbb{R}^{n\times d}$,

$$h(\boldsymbol{B}) = \lambda \||\boldsymbol{B}| - \mathbf{1}\|_1 = \sum_{i=1}^{n}\sum_{j=1}^{d} \lambda \big||b_{ij}| - 1\big|,$$

its conjugate is separable element-wise:

$$h^*(\boldsymbol{\Lambda}) = \sum_{i=1}^{n}\sum_{j=1}^{d} h^*(\Lambda_{ij}), \text{ where } h^*(\Lambda_{ij}) = \begin{cases} |\Lambda_{ij}|, & \text{if } \Lambda_{ij} \in [-\lambda, \lambda], \\ +\infty, & \text{otherwise,} \end{cases}$$

whose proximal operator in the piecewise solution (see Figure 4.1 for details) is

$$[\text{prox}_{\tau h^*}(\boldsymbol{Y})]_{ij} = \begin{cases} \lambda, & Y_{ij} > \lambda + \tau, \\ Y_{ij} - \tau, & \tau < Y_{ij} \le \lambda + \tau, \\ 0, & -\tau \le Y_{ij} \le \tau, \\ Y_{ij} + \tau, & -\lambda - \tau \le Y_{ij} < -\tau, \\ -\lambda, & Y_{ij} < -\lambda - \tau. \end{cases} \tag{4.3}$$

This motivates us to perform a proximal step to update $\boldsymbol{\Lambda}$.

We now outline the **DualHash** framework, which employs an alternating update scheme for $\boldsymbol{x}, \boldsymbol{B}, \boldsymbol{\Lambda}$.

First, we adopt a stochastic first-order method for the $\boldsymbol{x}$-update since computing the full gradient $\nabla_{\boldsymbol{x}} F(\boldsymbol{x}, \boldsymbol{B})$ is computationally expensive when the number of samples n is large, as is typical in deep learning. Specifically, at each iteration k, we update $\boldsymbol{x}$ by utilizing a stochastic gradient descent step

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k - \eta_k \tilde{\nabla}_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k),$$

where $\tilde{\nabla}_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k)$ is a stochastic estimator of $\nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k)$.

Second, the auxiliary variable $\boldsymbol{B}$ is updated via a vanilla gradient descent step (4.4); this suffices because the objective (4.2) with respect to $\boldsymbol{B}$ is smooth with an inexpensive gradient,

$$\boldsymbol{B}^{k+1} = \boldsymbol{B}^k - \tau \nabla_{\boldsymbol{B}} \mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) = \boldsymbol{B}^k - \tau \left(\nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k) + \boldsymbol{\Lambda}^k \right). \tag{4.4}$$

Algorithm 4.1 DualHash-StoM

Initialize: $\boldsymbol{x}^1 = \boldsymbol{z}^1 = \boldsymbol{y}^1 \in \mathbb{R}^{d_{\boldsymbol{x}}}$, $\boldsymbol{B}^1 \in \mathbb{R}^{n\times d}$, $\boldsymbol{\Lambda}^1 \in \mathbb{R}^{n\times d}$, parameters $\{\eta_k\}$, τ, b_k, $\{\alpha_k, \beta_k \in (0,1)\}$
for $k = 1, 2, \ldots$ **do**
  Compute $\boldsymbol{G}^k$ by (4.6)
  Update $\boldsymbol{x}^{k+1}$ by (4.7)
  Update $\boldsymbol{B}^{k+1}$ by (4.4)
  Update $\boldsymbol{\Lambda}^{k+1}$ by (4.5)
end for
Return: $\boldsymbol{x}_R$ where R is uniformly sampled from $\{2, \ldots, T+1\}$

Finally, the dual variable $\boldsymbol{\Lambda}$ is updated via proximal gradient ascent on h^*, using an extrapolation step introduced in [10]. The update solves the following optimization problem:

$$\begin{aligned}\boldsymbol{\Lambda}^{k+1} &= \operatorname{prox}_{(1/\tau h^*)}\left(\boldsymbol{\Lambda}^k + \left(2\boldsymbol{B}^{k+1} - \boldsymbol{B}^k\right)\right)\\ &= \arg\min_{\boldsymbol{\Lambda}\in\mathbb{R}^{n\times d}} \left\{h^*(\boldsymbol{\Lambda}) - \langle \boldsymbol{\Lambda}, 2\boldsymbol{B}^{k+1} - \boldsymbol{B}^k\rangle + \frac{\tau}{2}\|\boldsymbol{\Lambda} - \boldsymbol{\Lambda}^k\|^2\right\}.\end{aligned} \tag{4.5}$$

Our framework accommodates flexible choices of $\tilde{\nabla}_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k)$, enabling the incorporation of advanced momentum techniques to enhance both theoretical complexity and practical performance.

4.2. Implementations.

4.2.1. DualHash with Momentum Acceleration. We first propose a stochastic first-order algorithm with momentum acceleration for solving Problem (P). At each iteration k, we uniformly sample a mini-batch $\mathcal{J}_k \subseteq \{1, \cdots, n\}$ with $b_k = |\mathcal{J}_k| \leq n$, and compute $\tilde{\nabla}_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k) := \nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k; \mathcal{J}_k)$ evaluated at the extrapolated point $\boldsymbol{z}^k$:

$$\boldsymbol{G}^k = \nabla_{\boldsymbol{x}} F(\boldsymbol{z}^k, \boldsymbol{B}^k; \mathcal{J}_k), \quad \text{where } \boldsymbol{z}^k = \boldsymbol{x}^k + \beta_k(\boldsymbol{x}^k - \boldsymbol{x}^{k-1}). \tag{4.6}$$

The update of $\boldsymbol{x}^k$ is then:

$$\boldsymbol{x}^{k+1} = \boldsymbol{y}^k - \eta_k \boldsymbol{G}^k, \text{ where } \boldsymbol{y}^k = \boldsymbol{x}^k + \alpha_k(\boldsymbol{x}^k - \boldsymbol{x}^{k-1}). \tag{4.7}$$

The overall algorithm, DualHash-StoM, is detailed in Algorithm 4.1.

Remark 4.1. The momentum acceleration framework in (4.7) covers these two accelerations: Nesterov acceleration [43] corresponds to $\alpha_k = \beta_k \neq 0$; the heavy-ball method [46] is obtained with $\beta_k = 0$; vanilla SGD is recovered when $\alpha_k = \beta_k = 0$. In this paper, we set $\alpha_k, \beta_k \in (0,1)$ for all $k \geq 1$.

4.2.2. DualHash with Recursive Momentum. Alternatively, we consider the StoRM estimator [16] for the $\boldsymbol{x}$-update, which achieves variance reduction without requiring full-gradient or large-batch gradient computations. Given its practical effectiveness in deep learning

Algorithm 4.2 DualHash-StoRM

Initialize: $\boldsymbol{x}^1 \in \mathbb{R}^{d_{\boldsymbol{x}}}$, $\boldsymbol{B}^1 \in \mathbb{R}^{n\times d}$, $\boldsymbol{\Lambda}^1 \in \mathbb{R}^{n\times d}$, parameters $\{\eta_k\}$, τ, b_k, $\{\rho_k \in (0,1)\}$
for $k = 1, 2, \ldots$ **do**
  Compute $\boldsymbol{D}^k$ by (4.8)
  Update $\boldsymbol{x}^{k+1}$ by (4.9).
  Update $\boldsymbol{B}^{k+1}$ by (4.4).
  Update $\boldsymbol{\Lambda}^{k+1}$ by (4.5).
end for
Return: $\boldsymbol{x}_R$ where R is uniformly sampled from $\{2, \ldots, T+1\}$

[34], we construct a recursive momentum estimator $\boldsymbol{D}^k$ for $\tilde{\nabla}_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k)$ as follows:

$$\boldsymbol{D}^k = \begin{cases} \nabla_{\boldsymbol{x}} F(\boldsymbol{x}^1, \boldsymbol{B}^1; \mathcal{J}_1), & \text{if } k = 1, \\ (1-\rho_{k-1})\big(\boldsymbol{D}^{k-1} - \nabla_{\boldsymbol{x}} F(\boldsymbol{x}^{k-1}, \boldsymbol{B}^{k-1}; \mathcal{J}_{k-1})\big) + \nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k; \mathcal{J}_k), & \text{if } k > 1. \end{cases} \tag{4.8}$$

The update of $\boldsymbol{x}$ is then

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k - \eta_k \boldsymbol{D}^k. \tag{4.9}$$

The overall algorithm, DualHash-StoRM, is detailed in Algorithm 4.2.

5. Complexity analysis.

We now analyze the complexity of the proposed algorithms in terms of the number of stochastic first-order oracle (SFO) calls required to find an ε-approximate stochastic critical point of the Lagrangian function (4.2). Specifically, we seek a point satisfying

$$\mathbb{E}[\mathrm{dist}^2(\mathbf{0}, \partial \mathcal{L}(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*))] \le \varepsilon^2 \tag{5.1}$$

for a given $\varepsilon > 0$. We assume in this paper the set $\mathrm{crit}\mathcal{L} := \{(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*) \in \mathbb{R}^{d_{\boldsymbol{x}}} \times \mathbb{R}^{n\times d} \times \mathbb{R}^{n\times d} : \mathbf{0} \in \partial \mathcal{L}(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*)\}$ is nonempty. This focus is standard in non-convex stochastic first-order optimization analysis, as finding a global or even local minimizer for non-convex optimization problems is generally NP-hard. We limit the maximum number of iterations to a fixed integer, denoted by T with a little abuse of notation.

In the remainder of this paper, we impose the following assumptions.

Assumption 5.1. *(i) Assume that the objective of Problem (P) is lower bounded by J^*;*
(ii) Assume that $F(\boldsymbol{x}, \boldsymbol{B})$ is $L_F = \mathcal{O}(1)$-smooth with respect to $(\boldsymbol{x}, \boldsymbol{B})$ at $\boldsymbol{x} \in \mathbb{R}^{d_{\boldsymbol{x}}}$ and $\boldsymbol{B} \in \mathbb{R}^{n\times d}$;
(iii) Suppose index $i \sim$ Uniform$\{1, \ldots, n\}$. For any $\boldsymbol{B} \in \mathbb{R}^{n\times d}$, there exists a constant $\sigma^2 \ge 0$ such that $\mathbb{E}_i[\nabla_{\boldsymbol{x}} F_i(\boldsymbol{x}, \boldsymbol{b}_i)] = \nabla_{\boldsymbol{x}} F(\boldsymbol{x}, \boldsymbol{B})$ and $\mathbb{E}_i[\|\nabla_{\boldsymbol{x}} F_i(\boldsymbol{x}, \boldsymbol{b}_i) - \nabla_{\boldsymbol{x}} F(\boldsymbol{x}, \boldsymbol{B})\|^2] \le \sigma^2$.

Remark 5.1. The smoothness constant L_F remains $\mathcal{O}(1)$, as the penalty formulation cancels the potential $\mathcal{O}(\sqrt{n})$ scaling. A rigorous analysis is in Appendix B.

We first prove that the iterates generated by our algorithms are bounded, which is crucial for the subsequent convergence analysis.

Lemma 5.1. *Let $\{(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k)\}_{k\in\mathbb{N}}$ be generated by Algorithm 4.1 or 4.2. Under Assumption 5.1(i), the sequence $\{(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k)\}_{k\in\mathbb{N}}$ is bounded.*

Proof. For $\boldsymbol{\Lambda}^k$, by the closed-form proximal mapping in (4.3), $\boldsymbol{\Lambda}^{k+1} \in [-\lambda, \lambda]^{n\times d}$. Hence $\{\boldsymbol{\Lambda}^k\}$ is bounded.

For $(\boldsymbol{x}^k, \boldsymbol{B}^k)$, since $\{\boldsymbol{\Lambda}^k\}$ is bounded and J is coercive in $(\boldsymbol{x}, \boldsymbol{B})$ with $J(\boldsymbol{x}^k, \boldsymbol{B}^k) \geq J^*$ by Assumption 5.1(i), the sublevel set $\{(\boldsymbol{x}, \boldsymbol{B}) : J(\boldsymbol{x}, \boldsymbol{B}) \leq c\}$ is bounded for any $c < +\infty$, hence $\{(\boldsymbol{x}^k, \boldsymbol{B}^k)\}$ is bounded. ∎

We then prove that any primal-dual optimal solution $(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*)$ satisfies the first-order necessary optimality condition of the Lagrangian (4.2), i.e., it is a critical point of $\mathcal{L}$.

Lemma 5.2. *Under Assumption 5.1(i), suppose that$(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*)$ are the optimal solutions of Problem (P) and Problem (D), respectively. Then, it holds that $(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*) \in crit\mathcal{L}$, i.e., $\mathbf{0} \in \partial\mathcal{L}(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*)$.*

Proof. First, due to the coercivity of $\mathcal{L}(\boldsymbol{x}, \boldsymbol{B}, \boldsymbol{\Lambda})$, we have $\min\limits_{\boldsymbol{x},\boldsymbol{B}} \mathcal{L}(\boldsymbol{x}, \boldsymbol{B}, \boldsymbol{\Lambda}) > -\infty$. According to [6, Theorem 2.158], $(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*)$ are optimal solutions to the primal problem (P) and the dual problem (D) if and only if:

$$\begin{cases} (\boldsymbol{x}^*, \boldsymbol{B}^*) \in \operatorname{argmin}\limits_{\boldsymbol{x}\in\mathbb{R}^{d_{\boldsymbol{x}}}, \boldsymbol{B}\in\mathbb{R}^{n\times d}} \mathcal{L}(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*), \\ 0 = h(\boldsymbol{B}^*) + h^*(\boldsymbol{\Lambda}^*) - \langle \boldsymbol{\Lambda}^*, \boldsymbol{B}^* \rangle. \end{cases} \tag{5.2}$$

The first condition in (5.2) implies that $\mathbf{0}$ belongs to the subdifferential of $\mathcal{L}$ with respect to $(\boldsymbol{x}, \boldsymbol{B})$ at $(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*)$. Since $\mathcal{L}$ is continuously differentiable in $(\boldsymbol{x}, \boldsymbol{B})$, we have

$$\begin{cases} \mathbf{0} = \nabla f(\boldsymbol{x}^*) + \nabla_{\boldsymbol{x}} G(\boldsymbol{x}^*, \boldsymbol{B}^*), \\ \mathbf{0} = \nabla_{\boldsymbol{B}} G(\boldsymbol{x}^*, \boldsymbol{B}^*) + \boldsymbol{\Lambda}^*. \end{cases} \tag{5.3}$$

For the second condition in (5.2), by the definition of the conjugate function, if this equality holds, then $\boldsymbol{\Lambda}^*$ is the optimal solution of $\sup\{\langle \boldsymbol{b}^*, \boldsymbol{\lambda} \rangle - h^*(\boldsymbol{\lambda})\}$, i.e.,

$$\boldsymbol{B}^* \in \partial h^*(\boldsymbol{\Lambda}^*). \tag{5.4}$$

Combining (5.3) and (5.4), we obtain $\mathbf{0} \in \partial\mathcal{L}(\boldsymbol{x}^*, \boldsymbol{B}^*, \boldsymbol{\Lambda}^*)$, completing the proof. ∎

Our analysis aims to bound $\mathbb{E}[\|\partial\mathcal{L}\|^2]$. The presence of dual variables $\boldsymbol{\Lambda}$ renders the Lagrangian (4.2) non-descent, precluding the direct use of standard methods [62, 67]. To this end, we construct suitable Lyapunov functions (see (5.12) and (5.21)) to obtain the approximate descent property. We first analyze the dual dynamics by establishing bounds on $\|\boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k\|^2$ in terms of the primal residuals, enabling the Lyapunov construction.

Lemma 5.3. *Under Assumption 5.1, for any $k \geq 1$, it holds that*

$$\begin{aligned} &\|\boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k\|^2 \\ &\leq 3\tau^{-2}\|\boldsymbol{B}^{k+2} - \boldsymbol{B}^{k+1}\|^2 + 3(\tau^{-1} + L_F)^2\|\boldsymbol{B}^{k+1} - \boldsymbol{B}^k\|^2 + 3L_F^2\|\boldsymbol{x}^{k+2} - \boldsymbol{x}^{k+1}\|^2. \end{aligned} \tag{5.5}$$

Proof. Using the update of $\boldsymbol{B}^k$ in (4.4) twice yields that

$$\begin{aligned} &\|\boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k\| \\ &= \left\|\tau^{-1}(\boldsymbol{B}^{k+1} - \boldsymbol{B}^{k+2}) + \tau^{-1}(\boldsymbol{B}^{k+1} - \boldsymbol{B}^k) + \nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k) - \nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+2}, \boldsymbol{B}^{k+1})\right\| \\ &\leq \tau^{-1}\|(\boldsymbol{B}^{k+1} - \boldsymbol{B}^{k+2})\| + (\tau^{-1} + L_F)\|\boldsymbol{B}^{k+1} - \boldsymbol{B}^k\| + L_F\|\boldsymbol{x}^{k+2} - \boldsymbol{x}^{k+1}\|, \end{aligned}$$

where the second inequality uses the smoothness of $F(\boldsymbol{x}, \boldsymbol{B})$ in Assumption 5.1. Therefore, using the fact $(a+b+c)^2 \leq 3(a^2+b^2+c^2)$, one has

$$\begin{aligned} &\|\boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k\|^2 \\ &\leq 3\tau^{-2}\|\boldsymbol{B}^{k+2} - \boldsymbol{B}^{k+1}\|^2 + 3(\tau^{-1} + L_F)^2\|\boldsymbol{B}^{k+1} - \boldsymbol{B}^k\|^2 + 3L_F^2\|\boldsymbol{x}^{k+2} - \boldsymbol{x}^{k+1}\|^2, \end{aligned} \tag{5.6}$$

and completes the proof. ∎

Next, we analyze how the Lagrangian function (4.2) changes during the updates of primal and dual variables in one iteration. The total change can be decomposed as:

$$\begin{aligned} &\mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^{k+1}, \boldsymbol{\Lambda}^{k+1}) - \mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) \\ &= \underbrace{\mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^{k+1}, \boldsymbol{\Lambda}^{k+1}) - \mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k)}_{\text{one-iteration progress on } \boldsymbol{B} \text{ and } \boldsymbol{\Lambda}} + \underbrace{\mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) - \mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k)}_{\text{one-iteration progress on } \boldsymbol{x}}. \end{aligned} \tag{5.7}$$

Since the updates of $\boldsymbol{B}$ and $\boldsymbol{\Lambda}$ are common to Algorithms 4.1 and 4.2, the following lemma bounds their one-step change. For notational simplicity, we define

$$\Delta_{\boldsymbol{x}}^k = \frac{\|\boldsymbol{x}^k - \boldsymbol{x}^{k-1}\|^2}{2}, \quad \Delta_{\boldsymbol{B}}^k = \frac{\|\boldsymbol{B}^k - \boldsymbol{B}^{k-1}\|_F^2}{2}.$$

Lemma 5.4 (One-iteration progress on $\boldsymbol{B}$ and $\boldsymbol{\Lambda}$). *Consider the sequence $\{\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k\}_{k\in\mathbb{N}}$ generated by Algorithm 4.1 or Algorithm 4.2. Under Assumption 5.1(ii), for any $k \geq 1$, it holds that*

$$\begin{aligned} &\mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^{k+1}, \boldsymbol{\Lambda}^{k+1}) - \mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) \\ &\leq K_1\Delta_{\boldsymbol{B}}^{k+2} - K_2\Delta_{\boldsymbol{B}}^{k+1} + K_3\Delta_{\boldsymbol{B}}^k + K_4\Delta_{\boldsymbol{x}}^{k+2} + K_3\Delta_{\boldsymbol{x}}^{k+1}, \end{aligned} \tag{5.8}$$

where $K_1 = 3\delta\tau^{-1}$, $K_2 = 2\tau^{-1} - L_F - 3\delta\tau(\tau^{-1} + L_F)^2$, $K_3 = \frac{\tau L_F^2}{\delta}$, and $K_4 = 3\delta\tau L_F^2$.

Proof. Using the smoothness of F with respect to $\boldsymbol{B}$, we conclude that

$$\begin{aligned} &\mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^{k+1}, \boldsymbol{\Lambda}^{k+1}) - \mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) \\ &\leq -\left(\tau^{-1} - \frac{L_F}{2}\right)\|\boldsymbol{B}^{k+1} - \boldsymbol{B}^k\|^2 + \left\langle \boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k, \tau\left(\nabla_{\boldsymbol{B}} F(\boldsymbol{x}^k, \boldsymbol{B}^{k-1}) - \nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k)\right)\right\rangle. \end{aligned} \tag{5.9}$$

For the inner product term in (5.9), we apply the block-wise separability of (4.2) with respect to $\boldsymbol{B} = \{b_i\}_{i=1}^n$ and $\boldsymbol{\Lambda} = \{\lambda_i\}_{i=1}^n$ and obtain

$$\begin{aligned} &\langle \boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k, \tau(\nabla_{\boldsymbol{B}} F(\boldsymbol{x}^k, \boldsymbol{B}^{k-1}) - \nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k))\rangle \\ &\leq \frac{\delta\tau}{2}\|\boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k\|^2 + \frac{\tau}{2\delta}\|\nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k) - \nabla_{\boldsymbol{B}} F(\boldsymbol{x}^k, \boldsymbol{B}^{k+1})\|^2 \\ &\leq \frac{\delta\tau}{2}\|\boldsymbol{\Lambda}^{k+1} - \boldsymbol{\Lambda}^k\|^2 + \frac{\tau L_F^2}{2\delta}\left(\|\boldsymbol{x}^{k+1} - \boldsymbol{x}^k\|^2 + \|\boldsymbol{B}^k - \boldsymbol{B}^{k-1}\|^2\right), \end{aligned} \tag{5.10}$$

where the first inequality follows from the fact that $\langle \boldsymbol{a}, \boldsymbol{b}\rangle \leq \frac{\delta}{2}\|\boldsymbol{a}\|^2 + \frac{1}{2\delta}\|\boldsymbol{b}\|^2$ for any $\delta > 0$ and the second inequality follows from the smoothness of F with respect to $\boldsymbol{B}$ again. Then, substituting (5.10) and (5.6) into (5.9) and rearranging the inequality yields (5.8). ∎

We now analyze the stochastic $\boldsymbol{x}$-updates, which introduces algorithm-specific challenges: as (5.5) and (5.10) show, randomness in the $\boldsymbol{x}$-updates propagates to the $\boldsymbol{B}$- and $\boldsymbol{\Lambda}$-updates, necessitating careful parameter tuning and the construction of Lyapunov functions. We present the key lemmas and complexity results of both algorithms below[2]

5.1. Oracle complexity of DualHash-StoM. Let us denote the gradient error in the k-th iteration as

$$\boldsymbol{e}^k = \boldsymbol{G}^k - \nabla_{\boldsymbol{x}} F(\boldsymbol{z}^k, \boldsymbol{B}^k),$$

where $\boldsymbol{G}^k$ is defined in (4.6). Under Assumption 5.1(iii), one has $\mathbb{E}[\|\boldsymbol{e}^k\|^2] \le \sigma^2/b_k$. The one-iteration progress on $\boldsymbol{x}$ is estimated as follows, with the proof provided in Appendix C.1.

Lemma 5.5 (One-iteration progress on $\boldsymbol{x}$). *Consider the sequence $\{\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k\}_{k\in\mathbb{N}}$ generated by Algorithm 4.1. Under Assumption 5.1(ii) and the parameters in (5.18), for any $k \ge 1$, it holds that*

$$\mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) - \mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) \le -K_5 \Delta_{\boldsymbol{x}}^{k+1} + K_6 \Delta_{\boldsymbol{x}}^k + \frac{1}{L_F}\|\boldsymbol{e}^k\|^2, \tag{5.11}$$

where $K_5 = \frac{1-\alpha_k}{\eta_k} - 2L_F$ and $K_6 = \frac{\alpha_k}{\eta_k} + 2L_F\beta_k^2$.

To handle the oscillation of (4.2), we introduce the following Lyapunov function and sequence to establish an in-expectation approximate sufficient descent property for Algorithm 4.1:

$$\begin{aligned} \Psi_m(\mathcal{Q}^k) &= \mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) - C_1 \Delta_{\boldsymbol{B}}^{k+1} + C_2 \Delta_{\boldsymbol{B}}^k - C_3 \Delta_{\boldsymbol{x}}^{k+1} + C_4 \Delta_{\boldsymbol{x}}^k, \\ \mathcal{Q}^k &:= \{\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k, \boldsymbol{B}^{k+1}, \boldsymbol{B}^{k-1}, \boldsymbol{x}^{k+1}, \boldsymbol{x}^{k-1}\}, \quad \{\mathcal{Q}^k\}_{k\in\mathbb{N}}, \end{aligned} \tag{5.12}$$

where the positive coefficients C_1, C_2, C_3, C_4 (independent of k) are chosen as:

$$\begin{cases} C_1 &= K_1, \\ C_2 &= \frac{1}{2}(K_2 - K_1 + K_3), \\ C_3 &= K_4, \\ C_4 &= \frac{\bar{L}_F + \frac{2L_F\beta^2}{\alpha}}{1-2\alpha-\nu}, \quad \text{where } \bar{L}_F = 2L_F + K_3 + K_4. \end{cases} \tag{5.13}$$

We also assume the parameters $\{\eta_k\}$, $\{\alpha_k, \beta_k\}$, τ in Algorithm 4.1 are set as follows:

$$\eta_k = \frac{\eta}{L_F}, \quad \alpha_k = \alpha, \quad \beta_k = \beta, \quad \tau \le \frac{\sqrt{\tilde{\delta}}}{L_F}, \tag{5.14}$$

where $\eta = \frac{(1-2\alpha-\nu)/2}{\frac{L_F}{L_F} + \frac{(1+\nu)\beta^2}{(1-\alpha)\alpha}}$, $\alpha \in (0, \frac{1-\nu}{2})$, $\beta \in (0,1)$, $\tilde{\delta} = c_\delta \delta^2$ are given constants independent of T with some ν, δ, $c_\delta > 0$. The construction yields the following in-expectation approximate sufficient descent property of $\Psi_m(\mathcal{Q}^k)$. The proof is provided in Appendix C.2, and the positivity of the coefficients in (5.13) and the descent constants in (5.16) is established in Lemma C.2.

[2]Detailed proofs of all lemmas and theorems, along with complete parameter derivations, are given in Appendices C and D.

Lemma 5.6. *Under Assumption 5.1 and the parameter conditions (5.14), for any $k \geq 1$, it holds that*

$$\mathbb{E}[\Psi_m(\mathcal{Q}^{k+1})] - \mathbb{E}[\Psi_m(\mathcal{Q}^k)] \leq -C_{\boldsymbol{B}}\mathbb{E}[\Delta_{\boldsymbol{B}}^{k+1} + \Delta_{\boldsymbol{B}}^k] - C_{\boldsymbol{x}}\mathbb{E}[\Delta_{\boldsymbol{x}}^{k+1} + \Delta_{\boldsymbol{x}}^k] + \frac{\sigma^2}{b_k L_F}, \tag{5.15}$$

where the descent coefficients are given by

$$\begin{cases} C_{\boldsymbol{B}} &= C_2 - K_3 = \frac{K_2 - K_3 - K_1}{2} > 0, \\ C_{\boldsymbol{x}} &= \nu C_4 > 0. \end{cases} \tag{5.16}$$

We now present the convergence rate of Algorithm 4.1 under the parameter conditions (5.14), which guides the selection of the batch size b_k.

Lemma 5.7 (Convergence rate of Algorithm 4.1). *Under the conditions of Lemma 5.6, for any $T \geq 1$, there exists R uniformly selected from $\{2, \ldots, T+1\}$ such that $(\boldsymbol{x}^R, \boldsymbol{B}^R, \boldsymbol{\Lambda}^R)$ satisfies (5.1) with*

$$\mathbb{E}[\mathrm{dist}^2(0, \partial\mathcal{L}(\boldsymbol{x}^R, \boldsymbol{B}^R, \boldsymbol{\Lambda}^R))] \leq \frac{L_\Delta \Delta_1}{T} + C_\sigma \frac{\sigma^2}{b_k}, \tag{5.17}$$

where S_1, S_2, S_3, S_4 are defined in (C.10), $L_\Delta := \frac{\max\{S_1,S_2\}+\max\{S_3,S_4\}}{\min\{C_{\boldsymbol{B}}, C_{\boldsymbol{x}}\}}$, $C_\sigma := \frac{L_\Delta}{L_F} + 3$, and $\Delta_1 := \Psi_m^1 - \Psi_m^*$ denotes the initial objective gap with $\Psi_m^1 = \Psi_m(\mathcal{Q}^1)$ and the lower bound Ψ_m^*. Here L_Δ and C_σ are constants independent of T.

From (5.17), to achieve the $\mathcal{O}(T^{-1/2})$ convergence rate, we need to set $b_k = \Theta(T)$. Therefore, combining (5.14) with $b_k = b = c_b T$, the complete parameter setting in Algorithm 4.1 is

$$\eta_k = \frac{\eta}{L_F}, \quad \alpha_k = \alpha, \quad \beta_k = \beta, \quad \tau \leq \frac{\sqrt{\tilde{\delta}}}{L_F}, \quad b_k = b = c_b T, \tag{5.18}$$

where η, α, β, τ, $\tilde{\delta}$, c_b are given constants independent of T, and ν, δ, $c_\delta > 0$ are positive constants. The complexity result is established in Theorem 5.2, with the proof in Appendix C.4.

Theorem 5.2. *Under Assumption 5.1, let $\{\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k\}_{k\in\mathbb{N}}$ be the iterate sequence from Algorithm 4.1 with the parameters satisfying (5.18). Then there exists R uniformly selected from $\{2, \ldots, T+1\}$ such that $(\boldsymbol{x}^R, \boldsymbol{B}^R, \boldsymbol{\Lambda}^R)$ satisfies (5.1) provided that T satisfies*

$$T = \left\lceil \frac{L_\Delta \Delta_1 + C_\sigma \frac{\sigma^2}{c_b}}{\varepsilon^2} \right\rceil. \tag{5.19}$$

Moreover, the total number of SFO calls is of order $\mathcal{O}(\varepsilon^{-4})$.

5.2. Oracle complexity of DualHash-StoRM. Before proceeding, we introduce an additional assumption, also known as the mean-squared smoothness assumption, widely used in variance reduction methods [1, 23, 52, 20].

Assumption 5.2. *There exists $L_F > 0$ such that for any index i selected uniformly from $\{1, 2, \ldots, n\}$, $\boldsymbol{x}, \boldsymbol{x}' \in \mathbb{R}^{d_{\boldsymbol{x}}}$, and $\boldsymbol{B}, \boldsymbol{B}' \in \mathbb{R}^{n\times d}$ with $\boldsymbol{b}_i, \boldsymbol{b}_i' \in \mathbb{R}^d$ denoting the i-th rows of $\boldsymbol{B}, \boldsymbol{B}'$, it holds that*

$$\mathbb{E}_i\left[\|\nabla_{\boldsymbol{x}} F_i(\boldsymbol{x}, \boldsymbol{b}_i) - \nabla_{\boldsymbol{x}} F_i(\boldsymbol{x}', \boldsymbol{b}_i')\|^2\right] \leq L_F^2\left[\|\boldsymbol{x} - \boldsymbol{x}'\|^2 + \|\boldsymbol{B} - \boldsymbol{B}'\|^2\right].$$

This assumption is slightly stronger than the L_F-smoothness of the finite-sum function F in Assumption 5.1(ii). In particular, this assumption implies Assumption 5.1(ii) by Jensen's inequality, but not conversely.

Let us denote the gradient error

$$\boldsymbol{e}^k = \boldsymbol{D}^k - \nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k),$$

where $\boldsymbol{D}^k$ is defined in (4.8). Similarly, the following lemma estimates the one-iteration progress of the Lagrangian function (4.2) via the $\boldsymbol{x}$-update (4.9).

Lemma 5.8 (One-iteration progress on $\boldsymbol{x}$ [62]). *Consider the sequence $\{\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k\}_{k\in\mathbb{N}}$ generated by Algorithm 4.2. Under Assumption 5.1(i)-(ii), for any $k \geq 1$, it holds that*

$$\mathcal{L}(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) - \mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) \leq \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2 - (\eta_k^{-1} - L_F)\Delta_{\boldsymbol{x}}^{k+1}. \tag{5.20}$$

We introduce the following Lyapunov function of Algorithm 4.2:

$$\Psi_r(\mathcal{Q}^k) = \mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k) - C_1\Delta_{\boldsymbol{B}}^{k+1} + C_2\Delta_{\boldsymbol{B}}^k - C_3\Delta_{\boldsymbol{x}}^{k+1}, \tag{5.21}$$

where C_1, C_2, C_3, independent of k, are chosen as in (5.13). In the next lemma, we establish an approximate sufficient descent property of Ψ_r, with the proof provided in Appendix D.1.

Lemma 5.9. *Under Assumption 5.1(i)-(ii) and Assumption 5.2, with τ satisfying (5.14), for any $k \geq 1$, it holds that*

$$\Psi_r(\mathcal{Q}^{k+1}) - \Psi_r(\mathcal{Q}^k) \leq -C_{\boldsymbol{B}}\left(\Delta_{\boldsymbol{B}}^{k+1} + \Delta_{\boldsymbol{B}}^k\right) - C_{\boldsymbol{x}}^k\Delta_{\boldsymbol{x}}^{k+1} + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2, \tag{5.22}$$

where the descent coefficients are given by:

$$\begin{cases} C_{\boldsymbol{B}} &= C_2 - K_3 = \frac{K_2 - K_3 - K_1}{2} > 0, \\ C_{\boldsymbol{x}}^k &= \eta_k^{-1} - \tilde{L}_F, \ \text{where } \tilde{L}_F = L_F + K_3 + K_4. \end{cases}$$

Furthermore, the following inequalities hold:

$$\begin{cases} C_{\boldsymbol{B}}\left(\Delta_{\boldsymbol{B}}^{k+1} + \Delta_{\boldsymbol{B}}^k\right) \leq \Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1}) + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2, \\ C_{\boldsymbol{x}}^k\Delta_{\boldsymbol{x}}^{k+1} \leq \Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1}) + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2. \end{cases} \tag{5.23}$$

We now analysis the convergence rate of Algorithm 4.2 and we use the following parameter setting,

$$\eta_k = \frac{\eta}{\tilde{L}_F T^{\frac{1}{3}}}, \quad \rho_k = \frac{8\rho\eta^2}{T^{\frac{2}{3}}}, \quad \tau \leq \frac{\sqrt{\tilde{\delta}}}{L_F}, \quad b_1 = c_b T^{1/3}, \quad b_k = b \ (k \geq 2), \tag{5.24}$$

where η, ρ, $\tilde{\delta}$, c_b, and b are positive constants independent of T, and $\tilde{\delta} = c_\delta\delta^2$ with $\delta, c_\delta > 0$.

Lemma 5.10 (Convergence rate of Algorithm 4.2). *Under Assumption 5.1 and parameter conditions (5.24), for any $T \geq 1$, there exists R uniformly selected from $\{2, \ldots, T+1\}$ such that $(\boldsymbol{x}^R, \boldsymbol{B}^R, \boldsymbol{\Lambda}^R)$ satisfies (5.1) with*

$$\mathbb{E}[\mathrm{dist}^2(0, \partial\mathcal{L}(\boldsymbol{x}^R, \boldsymbol{B}^R, \boldsymbol{\Lambda}^R))] \leq \frac{\tilde{L}_\Delta \tilde{\Delta}_1}{T^{2/3}} + \tilde{C}_\sigma \left(\frac{\sigma^2}{4\rho\eta^2 b_1 T} + \frac{32\rho\eta^2\sigma^2}{b} \cdot \frac{1}{T^{2/3}} \right), \tag{5.25}$$

where $\tilde{S}_4 := \max\{S_1, S_2\}$ with S_1, S_2 defined in (C.10), $\tilde{L}_\Delta := \frac{\tilde{S}_4}{C_{\boldsymbol{B}}}\left(1 + \frac{8\eta^2}{\tilde{L}_F}\right) + \frac{2(C_\delta+4)\tilde{L}_F}{\eta} + (7 + 3C_\delta)$, $\tilde{C}_\sigma := 6 + C_\delta + \frac{\tilde{S}_4\eta}{2C_{\boldsymbol{B}}\tilde{L}_F}$, and $\tilde{\Delta}_1 := \Psi_r^1 - \Psi_r^*$ denotes the initial objective gap with $\Psi_r^1 = \Psi_r(\mathcal{Q}^1)$ and the lower bound Ψ_r^*. Here $\tilde{L}_\Delta$ and $\tilde{C}_\sigma$ are constants independent of T.

From (5.25), to acieve the $\mathcal{O}(T^{-2/3})$ convergence rate, we require $b_1 = \Theta(T^{1/3})$ and $b_k = \mathcal{O}(1)$ for $k > 1$. With this parameter choice, we proceed to analyze the oracle complexity of Algorithm 4.2.

Theorem 5.3. *Under Assumption 5.1(i), (iii) and Assumption 5.2, let $\{\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k\}_{k\in\mathbb{N}}$ be the iterate sequence from Algorithm 4.2 with the parameters satisfying (5.24). Then there exists R uniformly selected from $\{2, \ldots, T+1\}$ such that $(\boldsymbol{x}^R, \boldsymbol{B}^R, \boldsymbol{\Lambda}^R)$ satisfies (5.1) provided that T satisfies*

$$T = \left\lceil \frac{\left(\tilde{L}_\Delta \tilde{\Delta}_1 + \tilde{C}_\sigma \left(\frac{\sigma^2}{4\rho\eta^2 c_b} + \frac{32\rho\eta^2\sigma^2}{b}\right)\right)^{\frac{3}{2}}}{\varepsilon^3} \right\rceil, \tag{5.26}$$

Moreover, the total number of SFO calls is of order $\mathcal{O}(\varepsilon^{-3})$.

Remark 5.4. (i) The results in Theorems 5.2 and 5.3 are novel for stochastic, non-convex, multi-block primal-dual optimization, a challenging area where existing work has largely focused on one-block [62, 49, 53] or deterministic settings [70].

(ii) Algorithm 4.1 with constant parameters from (5.18) attains both oracle and sample $\mathcal{O}(\varepsilon^{-4})$ complexities, which matches that of stochastic first-order methods (SFOMs) for one-block stochastic optimization [25]. Meanwhile, Algorithm 4.2 with the refined constant parameters from (5.24) attains both oracle and sample $\mathcal{O}(\varepsilon^{-3})$ complexities. This matches the best-known lower bound [1] and the complexity of variance-reduced SFOMs for single-block problems [49, 62]. Notably, despite tackling a multi-block problem, our primal-dual framework preserves this efficiency without degrading the dependence on ε.

6. Numerical experiments.

We conduct extensive experiments to evaluate the effectiveness of DualHash against several state-of-the-art hashing methods on three standard image retrieval datasets. All experiments use PyTorch 1.12.1 with CUDA 11.3 on an NVIDIA V100 GPU platform.

6.1. Experimental Setup.

Datasets. **CIFAR-10**[3] is a **single-label** dataset comprising 60,000 32×32 color images from 10 classes. In our experiment, we randomly sample 1,000 images per class (10,000 total)

[3] https://www.cs.toronto.edu/~kriz/cifar.html

for training, 500 per class (5,000 total) for validation, and 500 per class (5,000 total) for testing. **NUS-WIDE**[4] is a **multi-label** dataset containing approximately 270,000 web images, each annotated with one or more labels from 81 categories. In our experiment, we use a subset of 195,834 images from the 21 most frequent concepts (each with at least 5,000 images) and randomly sample 700 images per class (14,700 total) for training, 300 per class (6,300 total) for validation, and 300 per class (6,300 total) for testing. **ImageNet**[5] is a large-scale image dataset from the Large Scale Visual Recognition Challenge (ILSVRC 2015). It contains over 1.2 million images in the training set and 50K images in the validation set, where each image is **single-labeled** by one of the 1,000 categories. Following [8], in our experiment, we randomly select 100 categories to create ImageNet-100. We use 100 images per category (10,000 total) for training, 30 per category (3,000 total) for validation, and 50 per category (5,000 total) for testing. The training, validation, and testing sets are mutually exclusive with no data overlap.

Training Setup. It is worth noting that our method is compatible with different similarity measurement paradigms, including pointwise, pairwise and tripletwise methods. In our experiments, we focus on deep supervised pairwise hashing using a pairwise cross-entropy loss f and the nonsmooth W-type regularization term $h(z) = \lambda\||\boldsymbol{z}| - 1\|_1$.

Baselines. We compare against eight baselines: SDH [48]; four deep hashing methods with W-type regularizations (DHN [71], DSH [39], DTSH [57], MDSHC [55]); and three state-of-the-art methods addressing quantization error (HashNet [8], DSDH [36], OrthoHash [28]). Following [39], we report the mean Average Precision (mAP), precision curves (P@topK), and precision within Hamming radius 2 (P@r=2). Detailed definitions and a full introduction of the baselines are provided in the Supplementary Materials A.

Architecture. For fair comparison, all deep learning-based methods employ AlexNet and ResNet-50[6] as the backbone network and use ELU activation to leverage its 1-Lipschitz continuous gradient property for satisfying the smoothness assumption (Assumption 5.1(ii)). All images are resized to 256×256 pixels and center-cropped to 224×224. We fine-tune the pre-trained convolutional layers (conv1–conv5) and fully connected layers (fc6–fc7), while initializing any newly added layers with Kaiming initialization[7]. For the non-deep method (SDH [48]), we use the following input features: 512-dimensional GIST vectors for CIFAR-10, 500-dimensional bag-of-words features for NUS-WIDE, and 1024-dimensional CNN features for ImageNet-100. During training, we monitor validation performance in real-time to observe convergence behavior and adjust hyperparameters accordingly. For evaluation, we use the training set as the retrieval database and the test set as queries.

Remark 6.1. For standard ReLU-activated backbones, existing nonsmooth stochastic optimization literature establishes convergence guarantees for such networks under weaker tame or weakly convex frameworks [18, 42], which are fundamentally distinct from our smooth primal-dual analysis. Extending our framework to this nonsmooth setting is an important direction for future work.

[4] http://lms.comp.nus.edu.sg/research/NUS-WIDE.htm

[5] https://www.image-net.org/

[6] AlexNet is the default backbone unless specified.

[7] Although DSDH [36] uses the CNN-F architecture, it shares a similar structure with AlexNet—both consist of five convolutional layers followed by two fully connected layers.

Table 6.1: The mAP results of DualHash and baselines with different numbers of bits on CIFAR-10 and NUS-WIDE datasets.

Methods	CIFAR-10 (mAP@All)				NUS-WIDE (mAP@5000)			
	16 bits	32 bits	48 bits	64 bits	16 bits	32 bits	48 bits	64 bits
SDH [48]	0.4254	0.4575	0.4751	0.4855	0.4756*	0.5545*	0.5786*	0.5812*
SDH-CNN	0.6665	0.6501	0.6484	0.6260	0.4872	0.6529	0.6451	0.5239
DSH [39]	0.6610	0.7530	0.7817	0.8010	0.5195	0.6629	0.6617	0.6634
DTSH [57]	0.7661	0.7501	0.7651	0.7744	0.6590	0.6845	0.7130	0.7371
DHN [71]	0.6568	0.7608	0.7838	0.8051	0.5562	0.6427	0.6808	0.7042
HashNet [8]	0.3354	0.6511	0.7732	0.8396	0.5879	0.6769	0.7221	0.7249
DSDH [36]	0.2732	0.3648	0.4222	0.4941	0.5153	0.5953	0.6338	0.6619
OrthoHash [28]	0.8070	0.8059	0.8387	0.8355	**0.6653**	0.6912	0.7083	0.7176
MDSHC [55]	0.8142	0.8238	0.8350	0.8262	0.5902	0.6452	0.6081	0.6493
DualHash-StoRM	0.8037	0.8051	0.8168	0.8345	0.6485	0.6802	0.6951	0.6982
DualHash-StoM	**0.8215**	**0.8481**	**0.8534**	**0.8539**	0.6339	**0.7002**	**0.7248**	**0.7448**

*SDH results on NUS-WIDE are cited from HashNet and reported for reference under a similar protocol. **Bold** values indicate the best results.

Hyperparameter Settings. Unless the original implementation imposes a method-specific component, all deep baselines are trained under the same data splits, evaluation protocol, backbone family, image preprocessing, and early-stopping rule as DualHash. All models are trained for a maximum of 300 epochs with early stopping based on validation mAP. We use a fixed mini-batch size of 256 for CIFAR-10 and 128 for NUS-WIDE and ImageNet-100. For DualHash-StoM, the learning rate η_k is selected from $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ with step decay, approximating $\eta_k \sim O(L_F^{-1})$ without computing L_F. For DualHash-StoRM, η_k is chosen from $\{5 \times 10^{-3}, 5 \times 10^{-2}, 5 \times 10^{-1}\}$ following $\mathcal{O}(T^{-1/3})$ scaling, and ρ_k is selected from $\{0.01, 0.05, 0.1, 0.2, 0.7\}$ via cross-validation. For DualHash-StoM, we set the momentum parameters $\alpha_k = \beta_k = 0.905$. Baselines follow their official optimizer settings. The proximal operator for $h^*(\cdot)$ requires $\tau < \lambda$ to ensure well-defined updates. We tune $\lambda \in \{10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$ and set τ accordingly. For all datasets, we set $\tau = 10^{-2}$, $\lambda = 5 \times 10^{-2}$, and $\gamma = 3$. In our practical implementation, we use a dual-step-size variant for multipliers $\mathbf{\Lambda}$, i.e., the $\mathbf{\Lambda}$-update uses step size 0.1τ, while the theoretical analysis is stated for the unified τ-scheme in Sections 4 and 5.

6.2. Convergence Validation. We provide empirical evidence that is consistent with our theoretical results (Theorems 5.2 and 5.3) and compare with HashNet, OrthoHash, and MDSHC with 64-bit codes on CIFAR-10 and NUS-WIDE. Figure 6.1 shows the training loss and validation mAP curves over epochs. As shown in Figure 6.1 (a)-(b), DualHash-StoRM achieves the fastest convergence and lowest loss values on CIFAR-10 and NUS-WIDE, requiring only approximately 25 epochs on CIFAR-10. This aligns with our theoretical outcome: the variance reduction mechanism accelerates convergence. Though slower than DualHash-StoRM, DualHash-StoM still outperforms other baselines (approximately 75 epochs on CIFAR-10). Meanwhile, Figure 6.1 (c)-(d) show that although DualHash-StoRM converges faster during

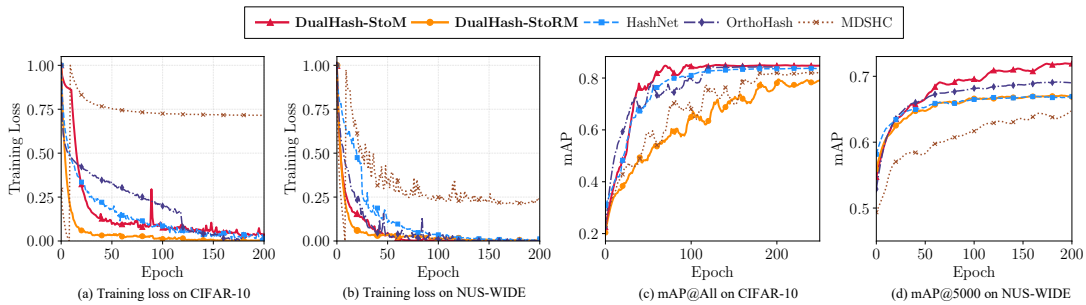


Figure 6.1: Training loss and the mAP results of DualHash and baselines for 64-bit codes on CIFAR-10 and NUS-WIDE datasets.

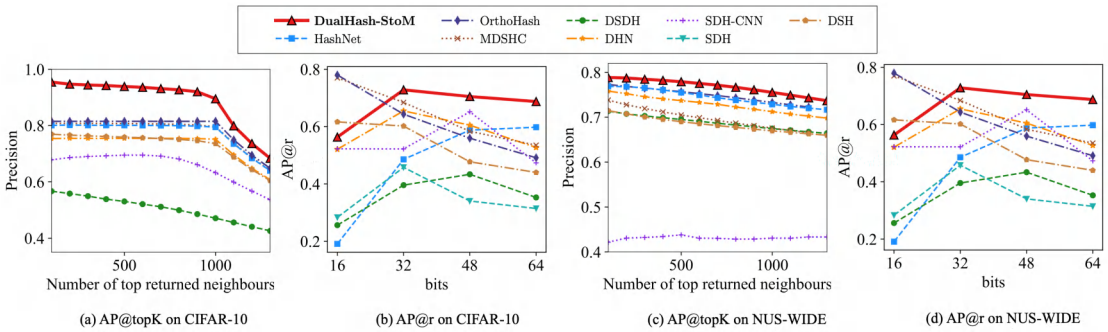


Figure 6.2: Retrieval performance of DualHash and baselines on CIFAR-10 and NUS-WIDE datasets. (a), (c): P@topK for 64-bit codes; (b), (d): P@r=2 with different numbers of bits.

Table 6.2: The mAP results of DualHash and baselines with different numbers of bits on CIFAR-10 and NUS-WIDE datasets with the ResNet-50 backbone.

	CIFAR-10 (mAP@All)				NUS-WIDE (mAP@5000)			
Method (ResNet-50)	16 bits	32 bits	48 bits	64 bits	16 bits	32 bits	48 bits	64 bits
DSH [39]	0.7403	0.8116	0.8217	0.8239	0.5559	0.6952	0.6981	0.7165
OrthoHash [28]	**0.9032**	0.9182	0.9202	0.9224	**0.7448**	0.7600	0.7569	0.7863
MDSHC [55]	0.8980	0.9007	0.9063	0.9227	0.6428	0.7002	0.7248	0.7758
DualHash-StoM	0.8559	**0.9202**	**0.9392**	**0.9416**	0.6538	**0.7783**	**0.7721**	**0.8016**

Bold values indicate the best results.

training, DualHash-StoM ultimately exhibits superior retrieval performance compared to other baselines.

6.3. Comparison of Retrieval Performance. We report retrieval performance on CIFAR-10 and NUS-WIDE datasets, as shown in Tables 6.1 and 6.2. In terms of mAP, DualHash methods

Table 6.3: The mAP results of DualHash and baselines with different numbers of bits on the ImageNet-100 dataset with the ResNet-50 backbone.

	ImageNet-100 (mAP@1000)			
Method (ResNet-50)	16 bits	32 bits	48 bits	64 bits
DSH [39]	0.7179	0.7448	0.7335	0.7585
OrthoHash [28]	0.8040	**0.8292**	0.8510	0.8636
MDSHC [55]	**0.8047**	0.8131	0.8561	0.8657
DualHash-StoM	0.7402	0.7705	**0.8662**	**0.8714**

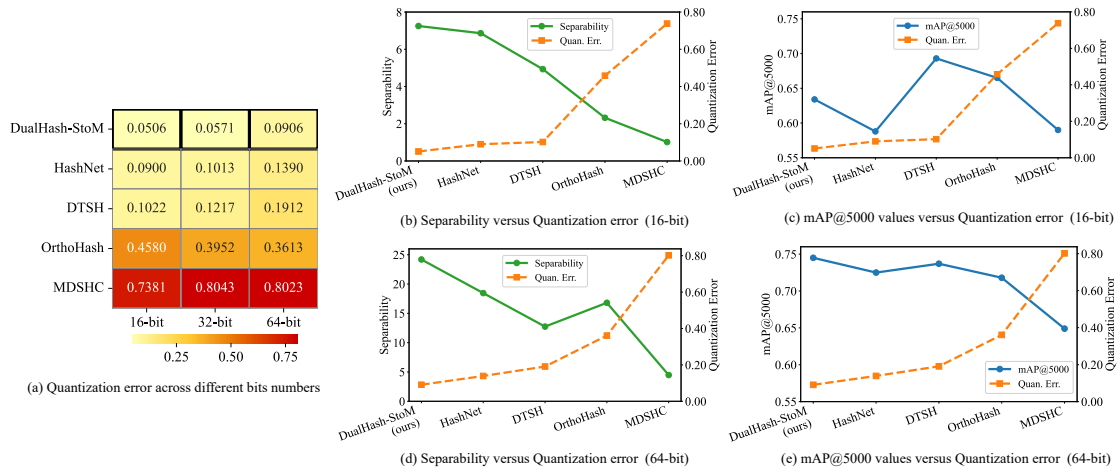
Bold values indicate the best results.

consistently outperform most baselines, including recent advanced methods like OrthoHash and MDSHC. On CIFAR-10, DualHash-StoM achieves 0.8539 mAP with 64-bit codes, outperforming OrthoHash (0.8355). On the NUS-WIDE dataset, it reaches 0.7448 mAP, significantly superior to MDSHC (0.6493). Although slightly less accurate than the StoM variant, DualHash-StoRM remains comparable to other baselines, providing a trade-off between accuracy and efficiency.

To further demonstrate the scalability of our approach, we evaluate our method and three baselines using a **ResNet-50 backbone**. As shown in Table 6.2, this modern architecture brings substantial performance gains in terms of mAP for all methods across CIFAR-10 and NUS-WIDE. Our DualHash-StoM, for example, achieves mAP improvements of 10.3% on CIFAR-10 (0.9416 vs 0.8539) and 7.6% on NUS-WIDE (0.8016 vs 0.7448) at 64 bits. Nevertheless, our method consistently maintains peak performance at 32 bits or more, outperforming all baseline methods. Moreover, on the more complex ImageNet-100 dataset, our method continues to demonstrate superior performance. As reported in Table 6.3, DualHash-StoM reaches 86.62% and 87.14% mAP at 48 and 64 bits, outperforming all baselines. These results confirm that our method maintains robust performance and strong generalization capability as data complexity increases.

For P@topK (Figure 6.2 (a),(c)), DualHash maintains 0.9 precision at lower top-k values on CIFAR-10, significantly outperforming other methods. This indicates that our method is suitable for precision-oriented image retrieval systems. For P@r=2 (Figure 6.2 (b),(d)), DualHash-StoM consistently maintains peak performance at 32 bits and above, with 32 bits being optimal across CIFAR-10 and NUS-WIDE. These results align with the fact that as bit length increases, the Hamming space becomes sparse and few data points fall within the Hamming ball with radius 2 [44].

6.4. Quantization Error Analysis. To elucidate the mechanisms behind the performance of DualHash, following [28], we examine the quantization error (the discrepancy between continuous outputs and binary codes, Figure 6.3) and the Hamming distance distributions of the resulting hash codes (Figures 6.4 and 6.5) across different bit lengths K on the NUS-WIDE dataset. We compare DualHash-StoM against four deep hashing methods: HashNet [8] and OrthoHash [28] (employing distinct quantization strategies), and DTSH [57] and MDSHC [55] (both utilizing W-type regularization).



(a) Quantization error across different bits numbers

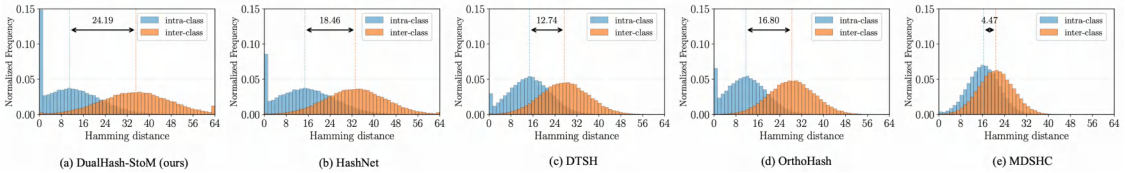
(b) Separability versus Quantization error (16-bit)

(c) mAP@5000 values versus Quantization error (16-bit)

(d) Separability versus Quantization error (64-bit)

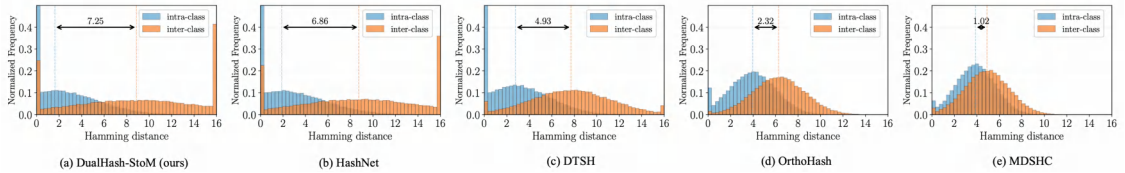
(e) mAP@5000 values versus Quantization error (64-bit)

Figure 6.3: Quantization error and its impact on retrieval performance across different numbers of bits on NUS-WIDE.



(a) DualHash-StoM (ours) (b) HashNet (c) DTSH (d) OrthoHash (e) MDSHC

Figure 6.4: Histogram of intra-class and inter-class Hamming distances of DualHash and baselines for 64-bit codes on the NUS-WIDE dataset. The arrow annotation is the separability in Hamming distances, $\mathbb{E}[D_{inter}] - \mathbb{E}[D_{intra}]$. We normalized the frequency so that the sum over all bins equals 1.



(a) DualHash-StoM (ours) (b) HashNet (c) DTSH (d) OrthoHash (e) MDSHC

Figure 6.5: Histogram of intra-class and inter-class Hamming distances of DualHash and baselines with 16-bit codes on NUS-WIDE.

(1) Quantization Error Correlates with Intra- and Inter-class Distances. We find that methods achieving smaller quantization errors produce hash codes with reduced intra-class distances and enhanced inter-class separability, as shown in Figure 6.3(a) and Figures 6.4 and 6.5. In the 64-bit setting (Figure 6.4), both HashNet and DualHash exhibit similar inter-class distributions (close to Hamming distance $K/2 = 32$). However, DualHash's lower quantization error (0.0906 vs. 0.1390) enables significantly superior separability, i.e., the difference in the mean of intra-class distance (the blue dotted line) with the mean of inter-class distance (the orange dotted line). In contrast, MDSHC's high quantization error (0.8023) leads

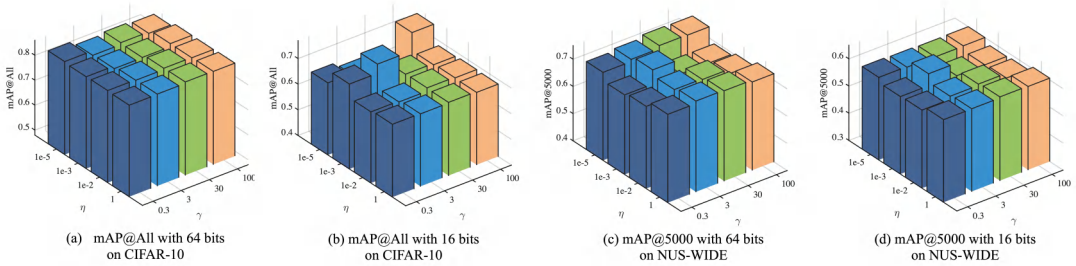


Figure 6.6: The mAP results of DualHash-StoM with different settings of η and λ for 64-bit and 16-bit codes on CIFAR-10 and NUS-WIDE.

to substantially larger intra-class distances and weaker inter-class separation, yielding a 5× inferior separability ratio (24.19 vs. 4.47). This demonstrates that reducing quantization error enhances code quality primarily by compressing intra-class distributions.

(2) DualHash Achieves Lower Quantization Error via Dual-Space Reformulation. As shown in Figure 6.3(a), DualHash attains the smallest quantization error across different bit lengths among all evaluated methods. This performance advantage stems from a fundamental methodological innovation: DualHash employs a novel dual-space reformulation of the W-regularization. The convexity property inherent in our dual formulation's conjugate function yields significant optimization advantages compared to directly optimizing the original nonconvex regularization or activation functions.

(3) Bit-Dependent Performance and the Separability-Semantics Trade-off. While lower quantization error generally improves separability (Figure 6.3(b) and (d)), its impact on retrieval performance varies significantly across bit settings, revealing a fundamental trade-off between class separability and semantic preservation.

▷ *High-bit regime ($K = 64$): Lower quantization error helps produce high mAP.* At $K = 64$ (Figures 6.3(d) and (e)), lower quantization error consistently correlates with higher mAP by enhancing separability and preserving both semantic discriminability and visual feature quality. Methods achieving low quantization loss (e.g., DualHash, HashNet, and DTSH) demonstrate superior retrieval accuracy, with DualHash achieving peak mAP performance.

▷ *Low-bit regime ($K = 16$): Low quantization error may produce low performance.* At $K = 16$ (Figure 6.3(b) and (c)), we observe a contrasting pattern: both extremely low and extremely high quantization error methods yield suboptimal performance. Notably, DTSH, which maintains intermediate quantization error, achieves superior mAP. Methods with very low quantization error (e.g., DualHash and HashNet) exhibit relatively lower mAP, particularly on multi-label datasets (e.g., NUS-WIDE). Specifically, we observe that DualHash generates hash codes with low intra-class distances and high inter-class separation. This creates relatively clean class separation, but the actual visual relationships between images may get lost. Consequently, DualHash excels on simple datasets like CIFAR-10 but shows relatively lower performance on complex datasets like NUS-WIDE (multi-label) and ImageNet-100 (large-scale single-label).

6.5. Parameter Sensitivity Analysis. To explore the sensitivity of key hyperparameters η and λ in DualHash, we conduct a grid search with $\eta \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ and $\lambda \in$

$\{10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$. Unlike many deep hashing methods that exhibit significant hyperparameter sensitivity, DualHash-StoM demonstrates robust performance across a wide range of hyperparameter combinations (see Figure 6.6). Specifically, 64-bit codes achieve excellent stability with mAP variations below 3% on CIFAR-10 and 8% on NUS-WIDE. While 16-bit codes show greater sensitivity (up to 18% on CIFAR-10), this behavior is consistent with the intrinsic optimization challenges of extremely compact hash codes. Overall, the method demonstrates strong stability for standard hash lengths ($\geq$64 bits), which simplifies hyperparameter tuning in practical high-precision retrieval applications.

7. Conclusion and Discussion. This paper proposed DualHash, a stochastic primal-dual algorithm for deep hashing with rigorous non-asymptotic convergence guarantees. Our approach effectively addressed the fundamental optimization challenge of W-type regularization with nonlinear composition through a novel "decoupling-dual-structure" strategy: transforming the nonconvex regularizer into a convex dual space with closed-form proximal updates. We provided a distinct theoretical framework beyond conventional primal-dual analysis, establishing complexity bounds of $\mathcal{O}(\varepsilon^{-4})$ for DualHash-StoM and improved $\mathcal{O}(\varepsilon^{-3})$ for DualHash-StoRM. Extensive experiments validated that DualHash achieved superior retrieval performance across all metrics, including the highest mAP at 32 bits or more, consistently lower quantization errors, and strong robustness on all three datasets (CIFAR-10, NUS-WIDE, and ImageNet-100).

Beyond deep hashing, our core **decoupling-dual-structure** optimization principle offered a promising pathway for tackling nonconvex regularization challenges in related domains, including image reconsrturction [19], plug-and-play proximal methods [59] and, binary neural networks [2].

References.

[1] Y. Arjevani, Y. Carmon, J. C. Duchi, D. J. Foster, N. Srebro, and B. Woodworth, *Lower bounds for non-convex stochastic optimization*, Mathematical Programming, 199 (2023), pp. 165–214.

[2] Y. Bai, Y.-X. Wang, and E. Liberty, *ProxQuant: Quantized neural networks via proximal operators*, in International Conference on Learning Representations (ICLR), 2019.

[3] F. Bian, J. Liang, and X. Zhang, *A stochastic alternating direction method of multipliers for non-smooth and non-convex optimization*, Inverse Problems, 37 (2021).

[4] F. Bian, R. Liu, and X. Zhang, *A stochastic three-block alternating minimization algorithm and its application to quantized deep neural networks*, CSIAM Transactions on Applied Mathematics, (2026).

[5] J. Bolte, S. Sabach, and M. Teboulle, *Nonconvex lagrangian-based optimization: Monitoring schemes and global convergence*, Mathematics of Operations Research, 43 (2018), pp. 1210–1232.

[6] J. F. Bonnans and A. Shapiro, *Perturbation Analysis of Optimization Problems*, Springer Science & Business Media, 2013.

[7] Y. Cao, M. Long, B. Liu, and J. Wang, *Deep cauchy hashing for hamming space retrieval*, in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 1229–1237.

[8] Z. Cao, M. Long, J. Wang, and P. S. Yu, *HashNet: Deep learning to hash by continuation*, in Proceedings of the IEEE International Conference on Computer Vision

(ICCV), 2017, pp. 5608–5617.

[9] A. Chambolle, M. J. Ehrhardt, P. Richtárik, and C.-B. Schönlieb, *Stochastic primal-dual hybrid gradient algorithm with arbitrary sampling and imaging applications*, SIAM Journal on Optimization, 28 (2018), pp. 2783–2808.

[10] A. Chambolle and T. Pock, *A first-order primal-dual algorithm for convex problems with applications to imaging*, Journal of Mathematical Imaging and Vision, 40 (2011), pp. 120–145.

[11] L. Chen, R. Liu, Y. Zhou, X. Ma, Y. Chen, and D. Zhang, *Deep hashing with semantic hash centers for image retrieval*, ACM Transactions on Information Systems, 43 (2025), pp. 160:1–160:38.

[12] X. Chen, C. Cui, and D. Han, *Convergence of three-block ADMM for weakly convex optimization problems*, SIAM Journal on Imaging Sciences, 18 (2025), pp. 449–493.

[13] E. Chouzenoux, A. Contreras, J.-C. Pesquet, and M. Savanier, *Convergence results for primal-dual algorithms in the presence of adjoint mismatch*, SIAM Journal on Imaging Sciences, 16 (2023), pp. 1–34.

[14] E. Chouzenoux and J.-C. Pesquet, *A stochastic majorize-minimize subspace algorithm for online penalized least squares estimation*, IEEE Transactions on Signal Processing, 65 (2017), pp. 4770–4783.

[15] C. Clason, S. Mazurenko, and T. Valkonen, *Acceleration and Global Convergence of a First-Order Primal-Dual Method for Nonconvex Problems*, SIAM Journal on Optimization, 29 (2019), pp. 933–963.

[16] A. Cutkosky and F. Orabona, *Momentum-based variance reduction in non-convex SGD*, in Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 15236–15245.

[17] D. Davis and D. Drusvyatskiy, *Stochastic model-based minimization of weakly convex functions*, SIAM Journal on Optimization, 29 (2019), pp. 207–239.

[18] D. Davis, D. Drusvyatskiy, S. Kakade, and J. D. Lee, *Stochastic subgradient method converges on tame functions*, Foundations of Computational Mathematics, 20 (2020), pp. 119–154.

[19] Q. Ding, G. Chen, X. Zhang, Q. Huang, H. Ji, and H. Gao, *Low-dose ct with deep learning regularization via proximal forward-backward splitting*, Physics in Medicine and Biology, 65 (2020), p. 125009.

[20] D. Driggs, J. Tang, J. Liang, M. Davies, and C.-B. Schönlieb, *A stochastic proximal alternating minimization for nonsmooth and nonconvex optimization*, SIAM Journal on Imaging Sciences, 14 (2021), pp. 1932–1970.

[21] D. Drusvyatskiy and C. Paquette, *Efficiency of minimizing compositions of convex functions and smooth maps*, Mathematical Programming, 178 (2019), pp. 503–558.

[22] E. Esser, X. Zhang, and T. F. Chan, *A general framework for a class of first order primal-dual algorithms for convex optimization in imaging science*, SIAM Journal on Imaging Sciences, 3 (2010), pp. 1015–1046.

[23] C. Fang, C. J. Li, Z. Lin, and T. Zhang, *SPIDER: Near-optimal non-convex optimization via stochastic path-integrated differential estimator*, in Advances in Neural Information Processing Systems, vol. 31, 2018, pp. 687–697.

[24] Y. Gao and W. Zhang, *An alternative extrapolation scheme of PDHGM for saddle*

point problem with nonlinear function, Computational Optimization and Applications, 85 (2023), pp. 263–291.

[25] S. Ghadimi, G. Lan, and H. Zhang, *Mini-batch stochastic approximation methods for nonconvex stochastic composite optimization*, Mathematical Programming, 155 (2013), pp. 267 – 305.

[26] J. Guo, *Preconditioned primal-dual gradient methods for nonconvex composite and finite-sum optimization*, AIMS Mathematics, 11 (2026), pp. 2188–2226.

[27] C. He and H. Wei, *Mambahash: Visual state space deep hashing model for large-scale image retrieval*, in Proceedings of the International Conference on Multimedia Retrieval (ICMR), 2025, pp. 442–450.

[28] I. T. Hoe, K. W. Ng, T. Zhang, C. S. Chan, Y.-Z. Song, and T. Xiang, *One loss for all: Deep hashing with a single cosine similarity based learning objective*, in Advances in Neural Information Processing Systems (NeurIPS), 2021, pp. 24286–24298.

[29] Q. Hu, D. Zhu, and T. Yang, *Non-smooth weakly-convex finite-sum coupled compositional optimization*, in Advances in Neural Information Processing Systems (NeurIPS), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds., vol. 36, Curran Associates, Inc., 2023, pp. 5348–5403.

[30] Q.-Y. Jiang, X. Cui, and W.-J. Li, *Deep discrete supervised hashing*, IEEE Transactions on Image Processing, 27 (2018), pp. 5996–6009.

[31] L. Jin, J. Ma, Z. Liu, A. Gromov, A. Defazio, and L. Xiao, *Parq: Piecewise-affine regularized quantization*, in International Conference on Machine Learning (ICML), 2025, pp. 28044–28062.

[32] L. Jin and X. Wang, *A stochastic primal-dual method for a class of nonconvex constrained optimization*, Computational Optimization and Applications, 83 (2022), pp. 143–180.

[33] H. Lai, Y. Pan, Y. Liu, and S. Yan, *Simultaneous feature learning and hash coding with deep neural networks*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 3270–3278.

[34] K. Levy, A. Kavis, and V. Cevher, *STORM+: Fully adaptive SGD with recursive momentum for nonconvex optimization*, in Advances in Neural Information Processing Systems (NeurIPS), 2021, pp. 20571–20582.

[35] J. Li, C. Huang, R. Chan, H. Feng, M. K. Ng, and T. Zeng, *Spherical image inpainting with frame transformation and data-driven prior deep networks*, SIAM Journal on Imaging Sciences, 16 (2023), pp. 1177–1194.

[36] Q. Li, Z. Sun, R. He, and T. Tan, *Deep supervised discrete hashing*, in Advances in neural information processing systems (NeurIPS), 2017, pp. 2479–2488.

[37] W.-J. Li, S. Wang, and W.-C. Kang, *Feature learning based deep supervised hashing with pairwise labels*, in International Joint Conference on Artificial Intelligence (IJCAI), 2016, pp. 1711–1717.

[38] Z. Li and J. Li, *Simple and optimal stochastic gradient methods for nonsmooth nonconvex optimization*, Journal of Machine Learning Research, 23 (2022), pp. 1–61.

[39] H. Liu, R. Wang, S. Shan, and X. Chen, *Deep supervised hashing for fast image retrieval*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2064–2072.

[40] Z. Lu, S. Mei, and Y. Xiao, *Variance-reduced first-order methods for deterministically*

constrained stochastic nonconvex optimization with strong convergence guarantees, SIAM Journal on Optimization, 36 (2026), pp. 1–31.

[41] X. Luo, H. Wang, D. Wu, C. Chen, M. Deng, J. Huang, and X.-S. Hua, *A survey on deep hashing methods*, ACM Transactions on Knowledge Discovery from Data, 17 (2023).

[42] V. Mai and M. Johansson, *Convergence of a Stochastic Gradient Method with Momentum for Non-Smooth Non-Convex Optimization*, in International Conference on Machine Learning (ICML), vol. 119, 2020, pp. 6630–6639.

[43] Y. Nesterov, *A method for solving the convex programming problem with convergence rate O(1/k^2)*, Proceedings of the USSR Academy of Sciences, 269 (1983), pp. 543–547.

[44] M. Norouzi, A. Punjani, and D. J. Fleet, *Fast search in Hamming space with multi-index hashing*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2012, pp. 3108–3115.

[45] T. Pock and S. Sabach, *Inertial proximal alternating linearized minimization (iPALM) for nonconvex and nonsmooth problems*, SIAM Journal on Imaging Sciences, 9 (2016), pp. 1756–1787.

[46] B. T. Polyak, *Some methods of speeding up the convergence of iteration methods*, USSR Computational Mathematics and Mathematical Physics, (1964).

[47] T. R. Rockafellar and R. J.-B. Wets, *Variational Analysis*, Grundlehren der mathematischen Wissenschaften, Springer-Verlag, 1998.

[48] F. Shen, C. Shen, W. Liu, and H. T. Shen, *Supervised discrete hashing*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 37–45.

[49] Q. Shi, X. Wang, and H. Wang, *A momentum-based linearized augmented lagrangian method for nonconvex constrained stochastic optimization*, Mathematics of Operations Research, 51 (2025), pp. 92–133.

[50] Z. Shumaylov, J. Budd, S. Mukherjee, and C.-B. Schönlieb, *Weakly convex regularisers for inverse problems: Convergence of critical points and primal-dual optimisation*, in International Conference on Machine Learning (ICML), 2024.

[51] S. Su, C. Zhang, K. Han, and Y. Tian, *Greedy hash: Towards fast optimization for accurate hash coding in cnn*, in Advances in Neural Information Processing Systems (NeurIPS), vol. 31, 2018.

[52] Q. Tran Dinh, D. Liu, and L. Nguyen, *Hybrid variance-reduced SGD algorithms for minimax problems with nonconvex-linear function*, in Advances in Neural Information Processing Systems (NIPS), vol. 33, Curran Associates, Inc., 2020, pp. 11096–11107.

[53] Q. Tran-Dinh, N. H. Pham, D. T. Phan, and L. M. Nguyen, *A hybrid stochastic optimization framework for composite nonconvex optimization*, Mathematical Programming, 191 (2022), pp. 1005–1071.

[54] B. Wang and T. Yang, *A near-optimal single-loop stochastic algorithm for convex finite-sum coupled compositional optimization*, in International Conference on Machine Learning, vol. 267 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 65091–65121.

[55] L. Wang, Y. Pan, C. Liu, H. Lai, J. Yin, and Y. Liu, *Deep hashing with minimal-distance-separated hash centers*, in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 23455–23464.

[56] X. Wang and X. Chen, *Complexity of finite-sum optimization with nonsmooth composite functions and non-lipschitz regularization*, SIAM Journal on Optimization, 34 (2024), pp. 2472–2502.
[57] X. Wang, Y. Shi, and K. M. Kitani, *Deep supervised hashing with triplet labels*, in Asian Conference on Computer Vision (ACCV), 2016, pp. 70–84.
[58] Y. Wang, W. Yin, and J. Zeng, *Global convergence of ADMM in nonconvex nonsmooth optimization*, Journal of Scientific Computing, 78 (2019), p. 29–63.
[59] K. Wei, A. Aviles-Rivero, J. Liang, Y. Fu, H. Huang, and C.-B. Schönlieb, *TFPnP: Tuning-free plug-and-play proximal algorithms with applications to inverse imaging problems*, Journal of Machine Learning Research, 23 (2022), pp. 1–48.
[60] Y. Weiss, A. Torralba, and R. Fergus, *Spectral hashing*, in Advances in Neural Information Processing Systems (NeurIPS), 2008, pp. 1753–1760.
[61] R. Xia, Y. Pan, H. Lai, C. Liu, and S. Yan, *Supervised hashing for image retrieval via image representation learning*, in AAAI conference on artificial intelligence (AAAI), 2014, pp. 2156–2162.
[62] Y. Xu and Y. Xu, *Momentum-based variance-reduced proximal stochastic gradient method for composite nonconvex stochastic optimization*, Journal of Optimization Theory and Applications, 196 (2023), pp. 266–297.
[63] Y. Yao, Q. Lin, and T. Yang, *A note on complexity for two classes of structured non-smooth non-convex compositional optimization*, 2024, https://arxiv.org/abs/2411.14342.
[64] P. Yin, Y. Lou, Q. He, and J. Xin, *Minimization of 1-2 for compressed sensing*, SIAM Journal on Scientific Computing, 37 (2015), pp. A536–A563.
[65] L. Yuan, T. Wang, X. Zhang, F. E. Tay, Z. Jie, W. Liu, and J. Feng, *Central Similarity Quantization for Efficient Image and Video Retrieval*, in 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, jun 2020, IEEE, pp. 3080–3089.
[66] H. Zhang, N. Marumo, T. K. Pong, and A. Takeda, *Complexity and convergence analysis of a single-loop sdcam for lipschitz composite optimization and beyond*, 2025, https://arxiv.org/abs/2512.24059.
[67] J. Zhang and L. Xiao, *Stochastic variance-reduced prox-linear algorithms for nonconvex composite optimization*, Mathematical Programming, (2022), pp. 1–43.
[68] Z. Zhang and G. Lan, *Optimal methods for convex nested stochastic composite optimization*, Mathematical Programming, 212 (2025), pp. 1–48.
[69] X. Zhao, M. Bai, and M. K. Ng, *Nonconvex optimization for robust tensor completion from grossly sparse observations*, Journal of Scientific Computing, 85 (2020), p. 46.
[70] D. Zhu, L. Zhao, and S. Zhang, *A first-order primal-dual method for nonconvex constrained optimization based on the augmented lagrangian*, Mathematics of Operations Research, 49 (2024), pp. 125–150.
[71] H. Zhu, M. Long, J. Wang, and Y. Cao, *Deep hashing network for efficient similarity retrieval*, in AAAI Conference on Artificial Intelligence (AAAI), 2016, pp. 2415–2421.

Theoretical Appendix. This supplementary material provides useful definitions and propositions, necessary lemmas with detailed proofs, and the complete derivations for the main theoretical results and parameters presented in the primary work.

Appendix A. Preliminaries. We introduce the following definitions related to subdifferentials.

Definition A.1 (Subdifferentials [47]). *Let $f : \mathbb{R}^q \to (-\infty, +\infty]$ be a proper closed function.*

(i) For a given $\boldsymbol{x} \in \operatorname{dom} f$, the Fréchet subdifferential of f at $\boldsymbol{x}$, written $\hat{\partial} f(\boldsymbol{x})$, is the set of all vectors $\boldsymbol{u} \in \mathbb{R}^q$ that satisfy

$$\liminf_{\boldsymbol{y} \neq \boldsymbol{x}, \boldsymbol{y} \to \boldsymbol{x}} \frac{f(\boldsymbol{y}) - f(\boldsymbol{x}) - \langle \boldsymbol{u}, \boldsymbol{y} - \boldsymbol{x} \rangle}{\|\boldsymbol{y} - \boldsymbol{x}\|} \geq 0.$$

(ii) The limiting-subdifferential, or simply the subdifferential, of f at $\boldsymbol{x}$, written $\partial f(\boldsymbol{x})$, is defined through the following closure process

$$\partial f(\boldsymbol{x}) := \{\boldsymbol{u} \in \mathbb{R}^q : \exists \boldsymbol{x}^k \to \boldsymbol{x}, f(\boldsymbol{x}^k) \to f(\boldsymbol{x}), \boldsymbol{u}^k \in \hat{\partial} f(\boldsymbol{x}^k) \to \boldsymbol{u} \text{ as } k \to \infty\}.$$

Notionally, $\hat{\partial} f(\boldsymbol{x}) = \partial f(\boldsymbol{x}) = \emptyset$ for all $\boldsymbol{x} \notin \operatorname{dom}(f)$. From Definition A.1 it follows that $\hat{\partial} f(\boldsymbol{x}) \subset \partial f(\boldsymbol{x})$ for all $\boldsymbol{x} \in \operatorname{dom}(f)$. Moreover, $\hat{\partial} f(\boldsymbol{x})$ is closed convex while $\partial f(\boldsymbol{x})$ is merely closed. In particular, if f is convex, then

$$\hat{\partial} f(\boldsymbol{x}) = \partial f(\boldsymbol{x}) = \{\boldsymbol{d} \in \mathbb{R}^q \mid f(\boldsymbol{y}) \geq f(x) + \langle \boldsymbol{d}, \boldsymbol{y} - \boldsymbol{x} \rangle, \forall \boldsymbol{y} \in \mathbb{R}^q\}.$$

The following proposition lists several useful properties of subgradients.

Proposition A.2 (Properties of subdifferential[47]). *Let $f : \mathbb{R}^{q_1} \to (-\infty, \infty]$ and $g : \mathbb{R}^{q_2} \to (-\infty, \infty]$ be proper and lower semicontinuous, and let $h : \mathbb{R}^{q_1} \to \mathbb{R}$ be continuously differentiable. Then the following results hold:*

(i) For any $\boldsymbol{x} \in \mathbb{R}^{q_1}$, we have $\hat{\partial} f(\boldsymbol{x}) \subseteq \partial f(\boldsymbol{x})$. If, in addition, f is convex, we have $\hat{\partial} f(\boldsymbol{x}) = \partial f(\boldsymbol{x})$.

(ii) For $\boldsymbol{x} \in \mathbb{R}^{q_1}$ with $f(\boldsymbol{x}) < \infty$, it holds

$$\hat{\partial}(f + h)(\boldsymbol{x}) = \hat{\partial} f(\boldsymbol{x}) + \nabla h(\boldsymbol{x}) \quad \text{and}$$
$$\partial(f + h)(\boldsymbol{x}) = \partial f(\boldsymbol{x}) + \nabla h(\boldsymbol{x}).$$

(iii) If $\sigma(\boldsymbol{x}_1, \boldsymbol{x}_2) = f_1(\boldsymbol{x}_1) + f_2(\boldsymbol{x}_2)$, then for any $\bar{\boldsymbol{x}}_1, \bar{\boldsymbol{x}}_2$

$$\begin{pmatrix} \hat{\partial}_{\boldsymbol{x}_1} f_1(\bar{\boldsymbol{x}}_1) \\ \hat{\partial}_{\boldsymbol{x}_2} f_2(\bar{\boldsymbol{x}}_2) \end{pmatrix} \subseteq \hat{\partial} \sigma(\bar{\boldsymbol{x}}_1, \bar{\boldsymbol{x}}_2) \quad \text{and} \quad \begin{pmatrix} \partial_{\boldsymbol{x}_1} f_1(\bar{\boldsymbol{x}}_1) \\ \partial_{\boldsymbol{x}_2} f_2(\bar{\boldsymbol{x}}_2) \end{pmatrix} \subseteq \partial \sigma(\bar{\boldsymbol{x}}_1, \bar{\boldsymbol{x}}_2).$$

For Algorithm 4.1, the key relationships among iterates via the momentum acceleration framework in (4.7) are established as follows.

Proposition A.3 (Lemma 5.2 in [45]). *Let $\{\boldsymbol{x}^k\}_{k \in \mathbb{N}}$, $\{\boldsymbol{y}^k\}_{k \in \mathbb{N}}$, and $\{\boldsymbol{z}^k\}_{k \in \mathbb{N}}$ be sequences generated by (4.7) and (4.6). For all $k \in \mathbb{N}$, we have:*

(i) $\|\boldsymbol{x}^k - \boldsymbol{y}^k\|^2 = \alpha_k^2 \|\boldsymbol{x}^k - \boldsymbol{x}^{k-1}\|^2$,

(ii) $\|\boldsymbol{x}^k - \boldsymbol{z}^k\|^2 = \beta_k^2 \|\boldsymbol{x}^k - \boldsymbol{x}^{k-1}\|^2$,

(iii) $\|\boldsymbol{x}^{k+1}-\boldsymbol{y}^k\|^2 \geq (1-\alpha_k)\|\boldsymbol{x}^{k+1}-\boldsymbol{x}^k\|^2+\alpha_k(\alpha_k-1)\|\boldsymbol{x}^k-\boldsymbol{x}^{k-1}\|^2$.

Proposition A.3(iii) decomposes the extrapolation error $\|\boldsymbol{x}^{k+1}-\boldsymbol{y}^k\|^2$ into a weighted combination of consecutive squared displacements, which, for $\alpha_k \in [0,1)$, enables the telescoping descent structure essential to the Lyapunov-based convergence analysis.

Appendix B. Discussions on Assumption 5.1(ii) in Section 5. This section verifies that the smoothness constant L_F in Assumption 5.1(ii) is independent of the sample size n, which is essential for our convergence analysis.

For the smooth properties of mappings in our problem, we consider the nonlinear mapping $\mathcal{D}(\boldsymbol{x}) = (\boldsymbol{D}_1(\boldsymbol{x}),\dots,\boldsymbol{D}_n(\boldsymbol{x}))$ from $\mathcal{X} \subseteq \mathbb{R}^{d_{\boldsymbol{x}}}$ to $\mathbb{R}^{n\times d}$, and make the following assumption:

Assumption B.1 (Properties of $\mathcal{D}$). *Suppose there exist constants $L_{\mathcal{D}}$, $L_{\nabla\mathcal{D}} > 0$ such that for any $\boldsymbol{x},\boldsymbol{x}' \in \mathcal{X}$: $\|\mathcal{D}(\boldsymbol{x})-\mathcal{D}(\boldsymbol{x}')\| \leq L_{\mathcal{D}}\|\boldsymbol{x}-\boldsymbol{x}'\|$ and $\|\nabla\mathcal{D}(\boldsymbol{x})-\nabla\mathcal{D}(\boldsymbol{x}')\| \leq L_{\nabla\mathcal{D}}\|\boldsymbol{x}-\boldsymbol{x}'\|$.*

Combining Assumption 5.1(i) and Assumption B.1, we can establish that there exist constants $C_{\mathcal{D}}, G_{\mathcal{D}}, C_{\mathcal{D}B} > 0$ such that:

$$\|\mathcal{D}(\boldsymbol{x})\| \leq C_{\mathcal{D}},\ \|\nabla\mathcal{D}(\boldsymbol{x})\| \leq G_{\mathcal{D}},\ \text{and}\quad \|\mathcal{D}(\boldsymbol{x})-\boldsymbol{B}\| \leq C_{\mathcal{D}B}.$$

Based on these assumptions, we can establish the smoothness properties of the composite function F. First, we observe that $P(\boldsymbol{x},\boldsymbol{B})$ is $\frac{\gamma}{n}$-smooth with respect to $\boldsymbol{B}$ and $\frac{\gamma}{n}(L_{\nabla\mathcal{D}}(C_{\mathcal{D}}+C_{\boldsymbol{B}})+G_{\mathcal{D}}L_{\mathcal{D}})$-smooth with respect to $\boldsymbol{x}$. Combining these properties, $P(\boldsymbol{x},\boldsymbol{B})$ has an overall smoothness constant:

$$L_s = \frac{\gamma}{n}\sqrt{2\max\{L_{\nabla\mathcal{D}}^2C_{\mathcal{D}B}^2+(2G_{\mathcal{D}}^2+1)L_{\mathcal{D}}^2, 2G_{\mathcal{D}}^2+1\}} = \mathcal{O}(1).$$

Consequently, $F(\boldsymbol{x},\boldsymbol{B})$ is L_F-smooth with respect to $(\boldsymbol{x},\boldsymbol{B})$, where $L_F = \mathcal{O}(1)$. An important observation is that, despite $C_{\mathcal{D}}$, $G_{\mathcal{D}}$ and $C_{\mathcal{D}B}$ potentially being $\mathcal{O}(\sqrt{n})$, the overall smoothness constant L_F remains independent of the sample size n, which is crucial for our convergence analysis.

Appendix C. Complexity analysis of DualHash-StoM (Algorithm 4.1). We adopt the notation and intermediate results from Section 5. Let us introduce the notation that will be used throughout our analysis:

$$\begin{aligned}
\boldsymbol{\xi}^k &= \{\xi_j^k, j \in \mathcal{J}_k\}, && \text{(mini-batch at iteration } k)\\
\boldsymbol{\xi}^{[k]} &= \{\boldsymbol{\xi}^1,\boldsymbol{\xi}^2,\dots,\boldsymbol{\xi}^k\}, && \text{(history of mini-batches up to iteration } k)\\
\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\cdot] &= \mathbb{E}[\cdot|\boldsymbol{\xi}^{[k]}]. && \text{(conditional expectation given history)}
\end{aligned}$$

For $\boldsymbol{d}^k \in \partial\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k)$, let us denote $\boldsymbol{d}^k = (\boldsymbol{d}_{\boldsymbol{x}}^k,\boldsymbol{d}_{\boldsymbol{B}}^k,\boldsymbol{d}_{\boldsymbol{\Lambda}}^k)$ and

$$\begin{cases}
\boldsymbol{d}_{\boldsymbol{x}}^k = \nabla_{\boldsymbol{x}}F(\boldsymbol{x}^k,\boldsymbol{B}^k),\\
\boldsymbol{d}_{\boldsymbol{B}}^k = \nabla_{\boldsymbol{B}}F(\boldsymbol{x}^k,\boldsymbol{B}^k)+\boldsymbol{\Lambda}^k,\\
\boldsymbol{d}_{\boldsymbol{\Lambda}}^k = \boldsymbol{B}^k-\mathcal{G}_{\boldsymbol{\Lambda}}^k,
\end{cases}$$

where $\mathcal{G}_{\boldsymbol{\Lambda}}^k \in \partial h^*(\boldsymbol{\Lambda}^k)$. We first present a lemma concerning a property of a continuously differentiable function with a Lipschitz continuous gradient, followed by a proposition that

details the interrelations among the sequences $\{\boldsymbol{x}^k\}_{k\in\mathbb{N}}$, $\{\boldsymbol{y}^k\}_{k\in\mathbb{N}}$, $\{\boldsymbol{z}^k\}_{k\in\mathbb{N}}$ generated by the algorithm.

Lemma C.1. *Let $f(\boldsymbol{x}):\mathbb{R}^q\to\mathbb{R}$ be a continuously differentiable function with an L-Lipschitz continuous gradient. Then, for any $\boldsymbol{x}$, $\boldsymbol{y}$, $\boldsymbol{z}\in\mathbb{R}^q$ and any $\boldsymbol{x}^+\in\mathbb{R}^q$ defined by*

$$\boldsymbol{x}^+=\boldsymbol{y}-\eta\nabla f(\boldsymbol{z}),\ \ \eta>0,$$

it holds that

(C.1)
$$f(\boldsymbol{x}^+)\le f(\boldsymbol{x})+\langle\nabla f(\boldsymbol{x})-\nabla f(\boldsymbol{z}),\boldsymbol{x}^+-\boldsymbol{x}\rangle+\frac{L}{2}\|\boldsymbol{x}^+-\boldsymbol{x}\|^2+\frac{1}{2\eta}\|\boldsymbol{x}-\boldsymbol{y}\|^2-\frac{1}{2\eta}\|\boldsymbol{x}^+-\boldsymbol{y}\|^2.$$

Proof. By the definition of $\boldsymbol{x}^+$, we obtain that

$$\boldsymbol{x}^+=\operatorname*{argmin}_{\boldsymbol{u}\in\mathbb{R}^d}\{\langle\nabla f(\boldsymbol{z}),\boldsymbol{u}-\boldsymbol{y}\rangle+\frac{1}{2\eta}\|\boldsymbol{u}-\boldsymbol{y}\|^2\},$$

and hence, in particular, by taking $\boldsymbol{u}=\boldsymbol{x}$, we obtain

$$0\le\langle\nabla f(\boldsymbol{z}),\boldsymbol{x}-\boldsymbol{x}^+\rangle+\frac{1}{2\eta}\|\boldsymbol{x}-\boldsymbol{y}\|^2-\frac{1}{2\eta}\|\boldsymbol{x}^+-\boldsymbol{y}\|^2.$$

Invoking the descent lemma for a smooth function yields the result. ∎

C.1. Proof of Lemma 5.5.

Proof. Recalling the equivalent relation:

(C.2)
$$\mathcal{L}(\boldsymbol{x}^{k+1},\boldsymbol{B}^k,\boldsymbol{\Lambda}^k)-\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k)=F(\boldsymbol{x}^{k+1},\boldsymbol{B}^k)-F(\boldsymbol{x}^k,\boldsymbol{B}^k).$$

Then, applying the smoothness of F with respect to $\boldsymbol{x}$ and (C.1) yields

$$\begin{aligned}F(\boldsymbol{x}^{k+1},\boldsymbol{B}^k)\le&F(\boldsymbol{x}^k,\boldsymbol{B}^k)+\langle\nabla_{\boldsymbol{x}}F(\boldsymbol{x}^k,\boldsymbol{B}^k)-\boldsymbol{G}^k,\boldsymbol{x}^{k+1}-\boldsymbol{x}^k\rangle\\&+\frac{L_F}{2}\|\boldsymbol{x}^{k+1}-\boldsymbol{x}^k\|^2+\frac{1}{2\eta_k}\|\boldsymbol{x}^k-\boldsymbol{y}^k\|^2-\frac{1}{2\eta_k}\|\boldsymbol{x}^{k+1}-\boldsymbol{y}^k\|^2.\end{aligned}\tag{C.3}$$

For the inner product term in (C.3), using the facts that $\langle a,b\rangle\le\frac{s}{2}\|a\|^2+\frac{1}{2s}\|b\|^2$ for any $s>0$ and $\|a-c\|^2\le2\|a-b\|^2+2\|b-c\|^2$ yields

$$\begin{aligned}&\langle\nabla_{\boldsymbol{x}}F(\boldsymbol{x}^k,\boldsymbol{B}^k)-\boldsymbol{G}^k,\boldsymbol{x}^{k+1}-\boldsymbol{x}^k\rangle\\&\le\frac{s}{2}\|\boldsymbol{x}^{k+1}-\boldsymbol{x}^k\|^2+\frac{1}{s}\|\nabla_{\boldsymbol{x}}F(\boldsymbol{x}^k,\boldsymbol{B}^k)-\nabla_{\boldsymbol{x}}F(\boldsymbol{z}^k,\boldsymbol{B}^k)\|^2+\frac{1}{s}\|\nabla_{\boldsymbol{x}}F(\boldsymbol{z}^k,\boldsymbol{B}^k)-\boldsymbol{G}^k\|^2\\&\le\frac{s}{2}\|\boldsymbol{x}^{k+1}-\boldsymbol{x}^k\|^2+\frac{L_F^2}{s}\|\boldsymbol{x}^k-\boldsymbol{z}^k\|^2+\frac{1}{s}\|\nabla_{\boldsymbol{x}}F(\boldsymbol{z}^k,\boldsymbol{B}^k)-\boldsymbol{G}^k\|^2.\end{aligned}\tag{C.4}$$

Setting $s=L_F$ in (C.4) and substituting into (C.3), then applying Proposition A.3, we obtain

$$\begin{aligned}F(\boldsymbol{x}^{k+1},\boldsymbol{B}^k)\le&F(\boldsymbol{x}^k,\boldsymbol{B}^k)-(\frac{1-\alpha_k}{\eta_k}-2L_F)\frac{\|\boldsymbol{x}^{k+1}-\boldsymbol{x}^k\|^2}{2}\\&+(\frac{\alpha_k}{\eta_k}+2L_F\beta_k^2)\frac{\|\boldsymbol{x}^k-\boldsymbol{x}^{k-1}\|^2}{2}+\frac{1}{L_F}\|\nabla_{\boldsymbol{x}}F(\boldsymbol{z}^k,\boldsymbol{B}^k)-\boldsymbol{G}^k\|^2.\end{aligned}\tag{C.5}$$

Substituting (C.5) into (C.2) yields (5.11) and completes the proof. ∎

C.2. Proof of Lemma 5.6.

Proof. Substituting one-iteration progress (5.11) and (5.8) into (5.7) , we obtain

$$\begin{aligned}&\mathcal{L}(\boldsymbol{x}^{k+1},\boldsymbol{B}^{k+1},\boldsymbol{\Lambda}^{k+1})-\mathcal{L}(\boldsymbol{x}^{k},\boldsymbol{B}^{k},\boldsymbol{\Lambda}^{k})\\ \le\ & K_1\Delta_{\boldsymbol{B}}^{k+2}-K_2\Delta_{\boldsymbol{B}}^{k+1}+K_3\Delta_{\boldsymbol{B}}^{k}+K_4\Delta_{\boldsymbol{x}}^{k+2}-(K_5-K_3)\Delta_{\boldsymbol{x}}^{k+1}+K_6\Delta_{\boldsymbol{x}}^{k}+\frac{1}{L_F}\|\boldsymbol{e}^k\|^2.\end{aligned}\tag{C.6}$$

Applying the definition of Ψ_m from (5.12) to (C.6), rearranging, and taking the conditional expectation on both sides, we obtain

$$\begin{aligned}&\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Psi_m(\mathcal{Q}^{k+1})\right]-\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Psi_m(\mathcal{Q}^{k})\right]\\ \le&-(C_1-K_1)\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}[\Delta_{\boldsymbol{B}}^{k+2}]-(K_2-C_1-C_2)\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}[\Delta_{\boldsymbol{B}}^{k+1}]-(C_2-K_3)\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\Delta_{\boldsymbol{B}}^{k}]\\ &-(C_3-K_4)\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}[\Delta_{\boldsymbol{x}}^{k+2}]-(K_5-K_3-C_3-C_4)\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}[\Delta_{\boldsymbol{x}}^{k+1}]-(C_4-K_6)\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\Delta_{\boldsymbol{x}}^{k}]+\frac{\sigma^2}{b_kL_F}.\end{aligned}\tag{C.7}$$

Then substituting the values of coefficients in (5.13) and parameter conditions (5.14) into (C.7), we derive

$$\begin{aligned}&\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Psi_m(\mathcal{Q}^{k+1})\right]-\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Psi_m(\mathcal{Q}^{k})\right]\\ \le&-C_{\boldsymbol{B}}\left(\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Delta_{\boldsymbol{B}}^{k+1}\right]+\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Delta_{\boldsymbol{B}}^{k}\right]\right)-C_{\boldsymbol{x}}\left(\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Delta_{\boldsymbol{x}}^{k+1}\right]+\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Delta_{\boldsymbol{x}}^{k}\right]\right)+\frac{\sigma^2}{b_kL_F}.\end{aligned}$$

Taking the full expectation of both sides leads to (5.16), thereby completing the proof. ∎

The technical proof that the coefficients of Ψ_m in (5.13) and the descent constants $C_{\boldsymbol{B}}$, $C_{\boldsymbol{x}}$ are indeed positive is given as follows.

Lemma C.2 (Positivity of Lyapunov related coefficients). *If parameter conditions (5.14) hold, then the following inequalities are satisfied:*

$$\begin{cases}K_5-K_3-C_3-C_4\ge\nu C_4,\\ C_4-K_6\ge\nu C_4,\\ C_4>0,\\ C_{\boldsymbol{B}}>0,\\ C_2,\,C_1>0,\\ C_{\boldsymbol{x}}>0.\end{cases}$$

Proof. For $K_5-K_3-C_3-C_4$ and C_4-K_6 in (C.7), we obtain by straightforward computations by choosing some $\nu>0$,

$$\begin{aligned}K_5-K_3-C_3-C_4&=\frac{1-\alpha_k}{\eta_k}-\bar{L}_F-(1+\nu)C_4+\nu C_4\\ &=(1-\alpha_k)\left(\frac{1}{\eta_k}-\frac{\bar{L}_F+(1+\nu)C_4}{1-\alpha_k}\right)+\nu C_4\\ &\ge\nu C_4.\end{aligned}$$

where the last inequality follows from $\alpha_k = \alpha$ and $\frac{1}{\eta_k} = \frac{\bar{L}_F + (1+\nu)C_4}{1-\alpha}$. Then, one has

$$\begin{aligned} C_4 - K_6 &= C_4 - \frac{\alpha}{\eta_k} - 2L_F\beta_k^2 \\ &= \frac{1}{1-\alpha}\left((1-\alpha)C_4 - \alpha\bar{L}_F - (1+\nu)C_4\alpha - 2L_F\beta^2(1-\alpha)\right) \\ &\geq \frac{1}{1-\alpha}\left((1-2\alpha-\nu)C_4 - \alpha\left(\bar{L}_F + \frac{2L_F\beta^2}{\alpha}\right)\right) + \nu C_4 + \frac{2L_F\beta^2\alpha}{1-\alpha} \\ &\geq \nu C_4, \end{aligned}$$

where the last inequality follows from $\alpha \in (0, \frac{1-\nu}{2})$ and $C_4 = \frac{\bar{L}_F + \frac{2L_F\beta^2}{\alpha}}{1-2\alpha-\nu} > 0$. We rearrange $\frac{1}{\eta_k}$ using the definition of C_4 and obtain $\eta_k = \frac{\eta}{L_F}$ where $\eta = \frac{(1-2\alpha-\nu)/2}{\frac{\bar{L}_F}{L_F} + \frac{(1+\nu)\beta^2}{(1-\alpha)\alpha}}$.

Next, we verify that the coefficients of Ψ_m satisfying (5.13) and τ satisfying (5.14) ensure that $C_{\boldsymbol{B}}, C_{\boldsymbol{x}} > 0$. Let us denote $t = \tau L_F$. Choosing $\delta > 0$, we have

$$C_{\boldsymbol{B}} = \frac{L_F}{2}\left(\frac{2-6\delta}{t} - \left(3\delta + \frac{1}{\delta}\right)t - (6\delta+1)\right) \geq 0.$$

If we choose $\delta = \frac{1}{6} > 0$, then

$$\frac{2}{L_F}C_{\boldsymbol{B}} = \frac{2}{t} - \frac{13}{2}t - 2 \geq 0.$$

By selecting $\tau \in \left(0, 6(\frac{\sqrt{30}-2}{13})\delta\frac{1}{L_F}\right]$, it is sufficient to ensures $C_{\boldsymbol{B}} > 0$, and consequently, $C_2, C_1 > 0$. Therefore, for some $\delta > 0$, setting $\tilde{\delta} = c_\delta\delta^2$ and ensuring $\tau^2 L_F^2 \leq \tilde{\delta}$ guarantee positive coefficients for Ψ_m in (5.13) and $C_{\boldsymbol{B}}, C_{\boldsymbol{x}} > 0$. ∎

We then derive an upper error bound on $\mathrm{dist}^2(\mathbf{0}, \partial\mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k))$ based on

$$\mathrm{dist}^2(\mathbf{0}, \partial\mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k)) \leq \|\boldsymbol{d}^k\|^2 = \|\boldsymbol{d}_{\boldsymbol{x}}^k\|^2 + \|\boldsymbol{d}_{\boldsymbol{B}}^k\|^2 + \|\boldsymbol{d}_{\boldsymbol{\Lambda}}^k\|^2. \tag{C.8}$$

Lemma C.3 (Stationarity error bound). *Under Assumption 5.1, for any $k \geq 1$, it holds that*

$$\mathbb{E}[dist^2(\mathbf{0}, \partial\mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k))] \leq S_1\mathbb{E}[\Delta_{\boldsymbol{B}}^{k+1}] + S_2\mathbb{E}[\Delta_{\boldsymbol{B}}^k] + S_3^k\mathbb{E}[\Delta_{\boldsymbol{x}}^{k+1}] + S_4^k\mathbb{E}[\Delta_{\boldsymbol{x}}^k] + \frac{3\sigma^2}{b_k L_F}, \tag{C.9}$$

where the coefficients are given by

$$\begin{cases} S_1 &= 4\tau^{-2} + 12, \\ S_2 &= 4(1 + 3(1+\tau L_F)^2), \\ S_3^k &= 4(3\eta_k^{-2} + L_F^2 + 3(\tau L_F)^2), \\ S_4^k &= 6(L_F^2\beta_k^2 + 6\eta_k^{-2}\alpha_k^2). \end{cases} \tag{C.10}$$

Proof. For estimating $\|\boldsymbol{d}_{\boldsymbol{x}}^k\|^2$, using the optimality condition of $\boldsymbol{x}^k$, we obtain

$$
\begin{aligned}
&\|\boldsymbol{d}_{\boldsymbol{x}}^k\|^2 \\
&= \|\nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k)\|^2 \\
&= \eta_k^{-2} \left\|\eta_k \nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k) + 0\right\|^2 \\
&= \eta_k^{-2} \left\|\eta_k \left(\nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k) - \boldsymbol{G}^k\right) + \boldsymbol{y}^k - \boldsymbol{x}^{k+1}\right\|^2 \\
&= \eta_k^{-2} \left\|\eta_k \left(\nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k) - \nabla_{\boldsymbol{x}} F(\boldsymbol{z}^k, \boldsymbol{B}^k) + \nabla_{\boldsymbol{x}} F(\boldsymbol{z}^k, \boldsymbol{B}^k) - \boldsymbol{G}^k\right) + \boldsymbol{y}^k - \boldsymbol{x}^k + \boldsymbol{x}^k - \boldsymbol{x}^{k+1}\right\|^2 \qquad \text{(C.11)} \\
&\le 3\|\nabla_{\boldsymbol{x}} F(\boldsymbol{x}^k, \boldsymbol{B}^k) - \nabla_{\boldsymbol{x}} F(\boldsymbol{z}^k, \boldsymbol{B}^k)\|^2 + 3\eta_k^{-2}\|\boldsymbol{x}^{k+1} - \boldsymbol{x}^k + \boldsymbol{x}^k - \boldsymbol{y}^k\|^2 + 3\|\boldsymbol{e}^k\|^2 \qquad \text{(C.12)} \\
&\le 3L_F^2\|\boldsymbol{x}^k - \boldsymbol{z}^k\|^2 + 6\eta_k^{-2}\left(\|\boldsymbol{x}^{k+1} - \boldsymbol{x}^k\|^2 + \|\boldsymbol{x}^k - \boldsymbol{y}^k\|^2\right) + 3\|\boldsymbol{e}^k\|^2, \qquad \text{(C.13)}
\end{aligned}
$$

where (C.11) is deduced by using the fact that $(a+b+c)^2 \le 3(a^2+b^2+c^2)$ and (C.12) is deduced by the smoothness of $F(\boldsymbol{x}, \boldsymbol{B})$ and $(a+b)^2 \le 2(a^2+b^2)$. Then applying Proposition A.3 to (C.13) concludes

$$
\|\boldsymbol{d}_{\boldsymbol{x}}^k\|^2 \le 6(L_F^2\beta_k^2 + 2\eta_k^{-2}\alpha_k^2)\Delta_{\boldsymbol{x}}^k + 12\eta_k^{-2}\Delta_{\boldsymbol{x}}^{k+1} + 3\|\boldsymbol{e}^k\|^2. \tag{C.14}
$$

For estimating $\|\boldsymbol{d}_{\boldsymbol{B}}^k\|^2$, using the $\boldsymbol{B}$-update (4.4) and the smoothness of F, we obtain

$$
\begin{aligned}
\|\boldsymbol{d}_{\boldsymbol{B}}^k\|^2 &= \|\nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k) + \boldsymbol{\Lambda}^k + \nabla_{\boldsymbol{B}} F(\boldsymbol{x}^k, \boldsymbol{B}^k) - \nabla_{\boldsymbol{B}} F(\boldsymbol{x}^{k+1}, \boldsymbol{B}^k)\|^2 \\
&\le 4\tau^{-2}\Delta_{\boldsymbol{B}}^{k+1} + 4L_F^2\Delta_{\boldsymbol{x}}^{k+1}, \qquad \text{(C.15)}
\end{aligned}
$$

where the last inequality follows from the fact $(a+b)^2 \le 2(a^2+b^2)$.

For estimating $\|\boldsymbol{d}_{\boldsymbol{\Lambda}}^k\|^2$, using the $\boldsymbol{\Lambda}$-update (4.5), we obtain

$$
\|\boldsymbol{d}_{\boldsymbol{\Lambda}}^k\|^2 \le 2\tau^2\|\boldsymbol{\Lambda}^k - \boldsymbol{\Lambda}^{k-1}\|^2 + 2\|\boldsymbol{B}^k - \boldsymbol{B}^{k-1}\|^2.
$$

Then, substituting (5.6) into the above inequality yields that

$$
\|\boldsymbol{d}_{\boldsymbol{\Lambda}}^k\|^2 \le 12\Delta_{\boldsymbol{B}}^{k+1} + 4(1 + 3(1+\tau L_F)^2)\Delta_{\boldsymbol{B}}^k + 12(\tau L_F)^2\Delta_{\boldsymbol{x}}^{k+1}. \tag{C.16}
$$

Finally, substituting (C.14), (C.15), and (C.16) into (C.8), and taking the full expectation on both sides, yields (C.9), completing the proof. ∎

C.3. Proof of Lemma 5.7.

Proof. From (C.9), we obtain

$$
\mathbb{E}[\mathrm{dist}^2(0, \partial\mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k))] \le (\max\{S_1, S_2\} + \max\{S_3, S_4\})\mathbb{E}[\Delta_{\boldsymbol{B}}^{k+1} + \Delta_{\boldsymbol{B}}^k + \Delta_{\boldsymbol{x}}^{k+1} + \Delta_{\boldsymbol{x}}^k] + \frac{3\sigma^2}{b_k L_F}. \tag{C.17}
$$

Similarly, from (5.15), we obtain

$$
\mathbb{E}[\Psi_m(\mathcal{Q}^{k+1})] - \mathbb{E}[\Psi_m(\mathcal{Q}^k)] \le -\min\{C_{\boldsymbol{B}}, C_{\boldsymbol{x}}\}\mathbb{E}[\Delta_{\boldsymbol{B}}^{k+1} + \Delta_{\boldsymbol{B}}^k + \Delta_{\boldsymbol{x}}^{k+1} + \Delta_{\boldsymbol{x}}^k] + \frac{\sigma^2}{b_k L_F},
$$

which yields that

$$\mathbb{E}[\Delta_{\boldsymbol{B}}^{k+1}+\Delta_{\boldsymbol{B}}^{k}+\Delta_{\boldsymbol{x}}^{k+1}+\Delta_{\boldsymbol{x}}^{k}] \leq \frac{1}{\min\{C_{\boldsymbol{B}}, C_{\boldsymbol{x}}\}}\left(\mathbb{E}[\Psi_m(\mathcal{Q}^k)]-\mathbb{E}[\Psi_m(\mathcal{Q}^{k+1})]+\frac{\sigma^2}{b_k L_F}\right)$$

Substituting this inequality into (C.17), summing over $k=1,\cdots,T$, and dividing by T, we obtain

$$\frac{1}{T}\sum_{k=1}^{T}\mathbb{E}[\mathrm{dist}^2(0,\partial\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k))] \leq \frac{L_\Delta}{T}\sum_{k=1}^{T}\left(\mathbb{E}[\Psi_m(\mathcal{Q}^k)]-\mathbb{E}[\Psi_m(\mathcal{Q}^{k+1})]\right)+(3+L_\Delta)\frac{\sigma^2}{b_k},$$

which yields the conclusion. Denoting $a_k=\mathbb{E}[\mathrm{dist}^2(0,\partial\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k))]$, we have

$$\frac{1}{T}\sum_{k=1}^{T}a_k \leq \frac{L_\Delta\Delta_1}{T}+(3+\frac{L_\Delta}{L_F})\frac{\sigma^2}{b_k}.$$

By uniformly selecting $R'\in\{1,\ldots,T\}$ and letting $R=R'+1$, we obtain

$$\mathbb{E}[\mathrm{dist}^2(0,\partial\mathcal{L}(\boldsymbol{x}^R,\boldsymbol{B}^R,\boldsymbol{\Lambda}^R))] \leq \frac{L_\Delta\Delta_1}{T}+(3+\frac{L_\Delta}{L_F})\frac{\sigma^2}{b_k},$$

and complete the proof. ∎

C.4. Proof of Theorem 5.2.

Proof. When $b_k=b=c_bT$, we have

$$\mathbb{E}[\mathrm{dist}^2(0,\partial\mathcal{L}(\boldsymbol{x}^R,\boldsymbol{B}^R,\boldsymbol{\Lambda}^R))] \leq \frac{1}{T}\left(L_\Delta\Delta_1+C_\sigma\frac{\sigma^2}{c_b}\right).$$

With the conditions on T in (5.19), this gives $\mathbb{E}[\mathrm{dist}^2(0,\partial\mathcal{L}(\boldsymbol{x}^R,\boldsymbol{B}^R,\boldsymbol{\Lambda}^R))]\leq\varepsilon^2$. Then, the oracle complexity of SFO is

$$\text{Complexity}=(T-1)b_k=\mathcal{O}(T^2)=\mathcal{O}(\varepsilon^{-4}).$$

∎

Appendix D. Complexity analysis of DualHash-StoRM (Algorithm 4.2).

We adopt the same notation as in Appendix C.

D.1. Proof of Lemma 5.9.

Proof. Substituting the one-iteration progress (5.20) and (5.9) into (5.7), we obtain

$$\begin{aligned}&\mathcal{L}(\boldsymbol{x}^{k+1},\boldsymbol{B}^{k+1},\boldsymbol{\Lambda}^{k+1})-\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k)\\ &\leq\frac{s}{2}\|\boldsymbol{e}^k\|^2-(-\frac{1}{s}+2\eta_k^{-1}-L_F-K_3)\Delta_{\boldsymbol{x}}^{k+1}+K_1\Delta_{\boldsymbol{B}}^{k+2}+K_4\Delta_{\boldsymbol{x}}^{k+2}-K_2\Delta_{\boldsymbol{B}}^{k+1}+K_3\Delta_{\boldsymbol{B}}^{k},\end{aligned}$$

where setting $s=\eta_k$ yields

$$\begin{aligned}&\mathcal{L}(\boldsymbol{x}^{k+1},\boldsymbol{B}^{k+1},\boldsymbol{\Lambda}^{k+1})-\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k)\\ &\leq\frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2-(\eta_k^{-1}-L_F-K_3)\Delta_{\boldsymbol{x}}^{k+1}+K_1\Delta_{\boldsymbol{B}}^{k+2}+K_4\Delta_{\boldsymbol{x}}^{k+2}-K_2\Delta_{\boldsymbol{B}}^{k+1}+K_3\Delta_{\boldsymbol{B}}^{k}.\end{aligned}\tag{D.1}$$

Applying the definition of Ψ_r from (5.21) into (D.1), we rearrange the above inequality

$$\begin{aligned}
&\Psi_r(\mathcal{Q}^{k+1}) - \Psi_r(\mathcal{Q}^k) \\
&\leq -(C_1 - K_1)\Delta_{\boldsymbol{B}}^{k+2} - (K_2 - C_1 - C_2)\Delta_{\boldsymbol{B}}^{k+1} - (C_2 - K_3)\Delta_{\boldsymbol{B}}^{k} - (C_3 - K_4)\Delta_{\boldsymbol{x}}^{k+2} \\
&\quad - (\eta_k^{-1} - L_F - K_3 - C_3)\Delta_{\boldsymbol{x}}^{k+1} - (C_4 - K_6)\Delta_{\boldsymbol{x}}^{k} + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2. \qquad \text{(D.2)}
\end{aligned}$$

By substituting the Lyapunov function coefficients (5.13) into (D.2), we obtain (5.22). With τ satisfying (5.14), Lemma C.2 guarantees that $C_{\boldsymbol{B}} > 0$. ∎

We provide a recursive bound on the gradient error vector $\{\boldsymbol{e}^k\}$ with analysis comparable to those in [62, Lemma 2] and [49, Lemma 8].

Lemma D.1 (Recursive bound on gradient error). *Under Assumption 5.1, for any $k \geq 1$, it holds that*

$$\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}[\|\boldsymbol{e}^{k+1}\|^2] \leq (1-\rho_k)^2\,\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\|\boldsymbol{e}^k\|^2] + \frac{2\rho_k^2\sigma^2}{|\mathcal{J}_{k+1}|} + \frac{4(1-\rho_k)^2L_F^2}{|\mathcal{J}_{k+1}|}\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Delta_{\boldsymbol{x}}^{k+1}\right].$$

We now establish a weighted error accumulation bound $\frac{1}{T}\sum_{k=1}^{T}\rho_k\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right]$ for the variance-reduced gradient estimator in Algorithm 4.2. For this analysis, we require the parameters η_k, ρ_k, τ and the batch size b_k to satisfy:

$$\eta_{k+1} \leq \eta_k, \quad \eta_k\tilde{L}_F \leq \frac{1}{2}, \quad 0 < 8\eta_1\eta_k L_F^2 \leq \rho_k \leq 1, \quad \tau \leq \frac{\sqrt{\tilde{\delta}}}{L_F}, \quad b_k = \begin{cases} b_1 & k = 1, \\ b & k \geq 2. \end{cases} \qquad \text{(D.3)}$$

Lemma D.2 (Weighted error accumulation bound). *Under Assumption 5.1 and parameter conditions (D.3), for any $T \geq 1$, the following weighted cumulative error bound holds:*

$$\frac{1}{T}\sum_{k=1}^{T}\rho_k\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] \leq \frac{16\eta_1L_F^2\tilde{\Delta}_1}{T} + \frac{2\sigma^2}{Tb_1} + \frac{1}{T}\sum_{k=2}^{T}\frac{4\rho_k^2\sigma^2}{b}, \qquad \text{(D.4)}$$

where $\tilde{\Delta}_1 = \Psi_r^1 - \Psi_r^*$, $\Psi_r^1 = \Psi_r(\mathcal{Q}^1)$ and Ψ_r^* is the lower bound of Ψ_r.

Proof. Under (D.3), we have a lower bound for $C_{\boldsymbol{x}}^k$ with

$$C_{\boldsymbol{x}}^k \geq \frac{1}{2\eta_1} > 0. \qquad \text{(D.5)}$$

Substituting this inequality into the second inequality of (5.23) and taking the conditional expectation on both sides yields

$$\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Psi_r(\mathcal{Q}^{k+1})\right] - \mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Psi_r(\mathcal{Q}^k)\right] \leq -\frac{1}{2\eta_1}\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Delta_{\boldsymbol{x}}^{k+1}\right] + \frac{\eta_k}{2}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right]. \qquad \text{(D.6)}$$

From Lemma D.1, we obtain

$$\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\|\boldsymbol{e}^{k+1}\|^2\right] \leq (1-\rho_k)\,\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] + \frac{2\rho_k^2\sigma^2}{|\mathcal{J}_{k+1}|} + 4(1-\rho_k)L_F^2\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Delta_{\boldsymbol{x}}^{k+1}\right],$$

where the inequality follows from $0 < \rho_k < 1$ and $|\mathcal{J}_{k+1}| \geq 1$, and this implies

$$\begin{aligned}
&- \mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Delta_{\boldsymbol{x}}^{k+1}\right] \\
&\leq \frac{1}{4(1-\rho_k)L_F^2}\left(\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] - \mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\|\boldsymbol{e}^{k+1}\|^2\right] - \rho_k \mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] + \frac{2\rho_k^2\sigma^2}{|\mathcal{J}_{k+1}|}\right).
\end{aligned}$$

Substituting this inequality into (D.6), we obtain

$$\begin{aligned}
&\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Psi_r(\mathcal{Q}^{k+1})\right] - \mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Psi_r(\mathcal{Q}^k)\right] \\
\leq &\frac{1}{8(1-\rho_k)\eta_1 L_F^2}\left(\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] - \mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\|\boldsymbol{e}^{k+1}\|^2\right] - \rho_k \mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] + \frac{2\rho_k^2\sigma^2}{|\mathcal{J}_{k+1}|}\right) + \frac{\eta_k}{2}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] \\
\leq &\frac{1}{8\eta_1 L_F^2}\left(\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right] - \mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\|\boldsymbol{e}^{k+1}\|^2\right] + \frac{2\rho_k^2\sigma^2}{|\mathcal{J}_{k+1}|}\right) - \left(\frac{\rho_k}{8\eta_1 L_F^2} - \frac{\eta_k}{2}\right)\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right],
\end{aligned} \tag{D.7}$$

where the second inequality follows from $1 - \rho_k \leq 1$. From (D.3), it follows that $\frac{\rho_k}{8\eta_1 L_F^2} - \frac{\eta_k}{2} \geq \frac{\rho_k}{16\eta_1 L_F^2}$. Applying this inequality to (D.7) yields the following expected descent property of the Lyapunov function Ψ_r

$$\begin{aligned}
&\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Psi_r(\mathcal{Q}^{k+1}) + \frac{1}{8\eta_1 L_F^2}\|\boldsymbol{e}^{k+1}\|^2\right] - \mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Psi_r(\mathcal{Q}^k) + \frac{1}{8\eta_1 L_F^2}\|\boldsymbol{e}^k\|^2\right] \\
\leq &-\frac{\rho_k}{16\eta_1 L_F^2}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\|\boldsymbol{e}^k\|^2] + \frac{\rho_k^2\sigma^2}{4\eta_1 L_F^2|\mathcal{J}_{k+1}|}.
\end{aligned} \tag{D.8}$$

Finally, summing up (D.8) from $k = 1$ to T and dividing by T yields

$$\frac{1}{T}\sum_{k=1}^{T}\rho_k \mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\|\boldsymbol{e}^k\|^2] \leq \frac{16\eta_1 L_F^2 \tilde{\Delta}_1}{T} + \frac{2\mathbb{E}_{\boldsymbol{\xi}^{[1]}}[\|\boldsymbol{e}^1\|^2]}{T} + \frac{1}{T}\sum_{k=2}^{T}\frac{4\rho_k^2\sigma^2}{|\mathcal{J}_{k+1}|},$$

yielding (D.4) and completes the proof. ∎

We derive an upper stationarity error bound for the iterates generated by Algorithm 4.2.

Lemma D.3 (Stationarity error bound). *Under Assumption 5.1 (ii), for any $k \geq 1$, it holds that*

$$dist^2(\mathbf{0}, \partial\mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k)) \leq S_1\Delta_{\boldsymbol{B}}^{k+1} + S_2\Delta_{\boldsymbol{B}}^{k} + S_3^k\Delta_{\boldsymbol{x}}^{k+1} + 2\|\boldsymbol{e}^k\|^2, \tag{D.9}$$

where the coefficients are defined as (C.10).

Proof. This proof follows a similar approach to that of Lemma C.3 in Appendix C. The key difference lies in the estimation of $\|\boldsymbol{d}_{\boldsymbol{x}}^k\|^2$. To estimate $\|\boldsymbol{d}_{\boldsymbol{x}}^k\|^2$, we use the optimality condition of $\boldsymbol{x}^{k+1}$ from (4.9) to obtain

$$\begin{aligned}
\|\boldsymbol{d}_{\boldsymbol{x}}^k\|^2 &= \eta_k^{-2}\left\|\eta_k\nabla_{\boldsymbol{x}}F(\boldsymbol{x}^k, \boldsymbol{B}^k) + 0\right\|^2 \\
&= \eta_k^{-2}\left\|\eta_k\left(\nabla_{\boldsymbol{x}}F(\boldsymbol{x}^k, \boldsymbol{B}^k) - \boldsymbol{D}^k\right) + \eta_k\boldsymbol{D}^k\right\|^2 \\
&= \eta_k^{-2}\left\|\eta_k\boldsymbol{e}^k + (\boldsymbol{x}^k - \boldsymbol{x}^{k+1})\right\|^2 \\
&\leq 4\eta_k^{-2}\Delta_{\boldsymbol{x}}^{k+1} + 2\|\boldsymbol{e}^k\|^2.
\end{aligned} \tag{D.10}$$

Then, substituting (D.10), (C.15), and (C.16) into (C.8) yields (D.9) and completes the proof. ■

Selecting appropriate parameters is crucial as the ultimate oracle complexities depend on these choices.

D.2. Proof of averaged stationarity error bound for STORM. We now establish the convergence rate of Algorithm 4.2 by taking constant parameters, i.e., $\eta_k \equiv \eta_1$ and $\rho_k \equiv \rho_1$ for $k \geq 1$. The first result is to estimate the averaged expected stationarity error.

Lemma D.4. *Under the conditions of Lemma D.2 , it holds that for any $T \geq 1$,*

$$\frac{1}{T}\sum_{k=1}^{T}\mathbb{E}[dist^2(\mathbf{0}, \partial\mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k))] \leq \frac{\tilde{\Delta}_1}{T}\left(\frac{\tilde{S}_4}{C_{\boldsymbol{B}}} + \frac{2(C_\delta^1+4)}{\eta_1} + \frac{(6+C_\delta^1+\frac{\tilde{S}_4\eta_1}{2C_{\boldsymbol{B}}})16\eta_1 L_F^2}{\rho_1}\right)$$
$$+\left(6+C_\delta^1+\frac{\tilde{S}_4\eta_1}{2C_{\boldsymbol{B}}}\right)\left(\frac{2\sigma^2}{b_1\rho_1 T}+\frac{4\rho_1\sigma^2(T-1)}{bT}\right), \tag{D.11}$$

where $\tilde{S}_4 = \max\{S_1, S_2\}$ and $C_\delta^1 = 1 + \frac{3\tilde{\delta}}{\tilde{L}_F^2}$.

Proof. Applying $\eta_k \leq \eta_1$ from (D.3) to (D.9) in Lemma D.3 yields

$$\text{dist}^2(\mathbf{0}, \partial\mathcal{L}(\boldsymbol{x}^k, \boldsymbol{B}^k, \boldsymbol{\Lambda}^k)) \leq \frac{S_4}{C_{\boldsymbol{B}}}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1})\right) + S_3^k\Delta_{\boldsymbol{x}}^{k+1} + (2+\frac{S_4\eta_1}{2C_{\boldsymbol{B}}})\|\boldsymbol{e}^k\|^2. \tag{D.12}$$

For the second term in (D.12), by recalling the second inequality in (5.23), we obtain

$$\Delta_{\boldsymbol{x}}^{k+1} \leq \frac{1}{C_{\boldsymbol{x}}^k}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1}) + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2\right),$$

where we use the positive value of $C_{\boldsymbol{x}}^k$ from (D.5). Therefore, substituting the inequality above and the definition of S_3^k from (C.10), into $S_3^k\Delta_{\boldsymbol{x}}^{k+1}$, one has

$$\begin{aligned}
&S_3^k\Delta_{\boldsymbol{x}}^{k+1}\\
&\leq \frac{S_3^k}{C_{\boldsymbol{x}}^k}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1}) + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2\right)\\
&\leq \frac{4[1+(1+3\tau^2)(\eta_k L_F)^2]}{\eta_k(1-\eta_k\tilde{L}_F)}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1}) + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2\right)\\
&\leq \frac{4[1+(1+\frac{3\tilde{\delta}}{L_F^2})(\eta_k L_F)^2]}{\eta_k(1-\eta_k\tilde{L}_F)}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1}) + \frac{\eta_k}{2}\|\boldsymbol{e}^k\|^2\right)\\
&\leq \frac{4[1+(1+\frac{3\tilde{\delta}}{\tilde{L}_F^2})(\eta_k \tilde{L}_F)^2]}{\eta_1(1-\eta_k\tilde{L}_F)}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1})\right) + \frac{2[1+(1+\frac{3\tilde{\delta}}{\tilde{L}_F^2})(\eta_k\tilde{L}_F)^2]}{1-\eta_k\tilde{L}_F}\|\boldsymbol{e}^k\|^2\\
&\leq \frac{4[1+C_\delta(\eta_k\tilde{L}_F)^2]}{\eta_1(1-\eta_k\tilde{L}_F)}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1})\right) + \frac{2[1+C_\delta(\eta_k\tilde{L}_F)^2]}{1-\eta_k\tilde{L}_F}\|\boldsymbol{e}^k\|^2\\
&\leq \frac{2(C_\delta+4)}{\eta_1}\left(\Psi_r(\mathcal{Q}^k) - \Psi_r(\mathcal{Q}^{k+1})\right) + (C_\delta+4)\|\boldsymbol{e}^k\|^2, \qquad \text{(D.13)}
\end{aligned}$$

where the third inequality follows from $\tau^2 \leq \tilde{\delta}/L_F^2$, the fourth inequality is derived from $\tilde{L}_F \geq L_F$ and $\eta_k \equiv \eta_1$, and the last inequality holds because $(1+cx^2)/(1-x) \leq 2+\frac{c}{2}$ for $0 < x \leq 1/2$ and constant $c > 1$ under parameter conditions (D.3).

On the other hand, by taking the conditional expectation on both sides of (D.12) and averaging over T, we obtain

$$\begin{aligned}
&\frac{1}{T}\sum_{k=1}^{T}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\mathrm{dist}^2(\mathbf{0},\partial\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k))\right]\\
\leq&\frac{S_4}{C_{\boldsymbol{B}}T}\sum_{k=1}^{T}\left(\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Psi_r(\mathcal{Q}^k)\right]-\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Psi_r(\mathcal{Q}^{k+1})\right]\right)+\frac{1}{T}\sum_{k=1}^{T}S_3^k\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\Delta_{\boldsymbol{x}}^{k+1}\right]\\
&+\frac{1}{T}\sum_{k=1}^{T}\left(2+\frac{S_4\eta_1}{2C_{\boldsymbol{B}}}\right)\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right]\\
\leq&\frac{\frac{S_4}{C_B}\tilde{\Delta}_1}{T}+\frac{1}{T}\sum_{k=1}^{T}S_3^k\mathbb{E}_{\boldsymbol{\xi}^{[k+1]}}\left[\Delta_{\boldsymbol{x}}^{k+1}\right]+\left(2+\frac{S_4\eta_1}{2C_{\boldsymbol{B}}}\right)\frac{1}{T}\sum_{k=1}^{T}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}\left[\|\boldsymbol{e}^k\|^2\right].
\end{aligned}\tag{D.14}$$

Then substituting (D.13) into (D.14) yields

$$\begin{aligned}
&\frac{1}{T}\sum_{k=1}^{T}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\mathrm{dist}^2(\mathbf{0},\partial\mathcal{L}(\boldsymbol{x}^k,\boldsymbol{B}^k,\boldsymbol{\Lambda}^k))]\\
\leq&\frac{\left(\frac{S_4}{C_B}+\frac{2(C_\delta+4)}{S_4\eta_1}\right)\tilde{\Delta}_1}{T}+\left(6+\frac{S_4\eta_1}{2C_{\boldsymbol{B}}}+C_\delta\right)\frac{1}{T}\sum_{k=1}^{T}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\|\boldsymbol{e}^k\|^2].
\end{aligned}\tag{D.15}$$

Since $\rho_k \equiv \rho_1$ for $k \geq 1$, then it follows from (D.4) that

$$\frac{1}{T}\sum_{k=1}^{T}\mathbb{E}_{\boldsymbol{\xi}^{[k]}}[\|\boldsymbol{e}^k\|^2] \leq \frac{16\eta_1 L_F^2\tilde{\Delta}_1}{\rho_1 T}+\frac{2\sigma^2}{b_1 T}+\frac{4\rho_1\sigma^2(T-1)}{bT}\tag{D.16}$$

Then, substituting (D.16) into (D.15) and taking the full expectation on both sides yields (D.11). ∎

D.3. Proof of Lemma 5.10.

Proof. Substituting the parameter conditions (5.24) into (D.11) from Lemma D.4, and applying the bound $\rho_k \equiv \frac{8\rho\eta^2}{T^{2/3}}$, $b_1 = c_b T^{1/3}$, $b_k = b$ for $k \geq 2$, we obtain (5.25) by selecting R uniformly from $\{2,\ldots,T+1\}$. ∎

With this parameter choice, we proceed to analyze the oracle complexity of Algorithm 4.2.

D.4. Proof of Theorem 5.3.

Proof. When $b_1 = c_b T^{1/3}$, we have

$$\mathbb{E}[\mathrm{dist}^2(0,\partial\mathcal{L}(\boldsymbol{x}^R,\boldsymbol{B}^R,\boldsymbol{\Lambda}^R)] \leq \frac{1}{T^{2/3}}\left(\tilde{L}_\Delta\tilde{\Delta}_1+\tilde{C}_\sigma\left(\frac{\sigma^2}{4\rho\eta^2 c_b}+\frac{32\rho\eta^2\sigma^2}{b}\right)\right).$$

With the conditions on T in (5.26), this gives $\mathbb{E}[\text{dist}^2(0, \partial\mathcal{L}(\boldsymbol{x}^R, \boldsymbol{B}^R, \boldsymbol{\Lambda}^R))] \leq \varepsilon^2$. Then, the oracle complexity is

$$\text{Complexity} = b_1 + (T-1)b = \mathcal{O}(T^{\frac{1}{3}}) + \mathcal{O}(T) = \mathcal{O}(T) = \mathcal{O}(\varepsilon^{-3}),$$

which completes the proof. ∎

Experimental Appendix.

This supplementary material provides detailed experimental settings and additional results.

Appendix E. Experiment Details.

E.1. Deep Pairwise Supervised Hashing Formulation. We consider deep supervised pairwise hashing in our experiments, which aims to learn binary codes that minimize Hamming distances for similar pairs and maximize them for dissimilar pairs. Let $\boldsymbol{A} = \{\boldsymbol{a}_i \in \mathbb{R}^{d_a} : i = 1, \ldots, n\}$ be the training set with labels $\mathcal{Y} = \{\boldsymbol{y}_i \in \{0,1\}^C\}$ (one-hot for single-label, multi-hot for multi-label). Key notations are in Table E.1.

Network Architecture. A CNN extracts features from $\boldsymbol{a}_i$, followed by a fully connected hash layer producing d-dimensional outputs $\boldsymbol{C}(\boldsymbol{x}; \boldsymbol{a}_i)$, which are passed through tanh to yield $\boldsymbol{u}_i$.

Loss Function. For hash codes in $\{-1,1\}^d$, the Hamming distance is $d_{ij}^h = \frac{1}{2}(d - \langle \boldsymbol{h}_i, \boldsymbol{h}_j \rangle)$. Following [61, 33], similarity $s_{ij}^o = 1$ if samples share labels, else 0. The likelihood is:

$$P(s_{ij}^o \mid \boldsymbol{h}_i, \boldsymbol{h}_j) = \sigma(\langle \boldsymbol{h}_i, \boldsymbol{h}_j \rangle)^{s_{ij}^o} \left(1 - \sigma(\langle \boldsymbol{h}_i, \boldsymbol{h}_j \rangle)\right)^{1 - s_{ij}^o},$$

where $\sigma(x) = (1 + \exp(-\alpha x))^{-1}$, $\alpha < 1$. Using continuous codes $\boldsymbol{u}_i$ and $s_{ij}^h = \frac{1}{2}\langle \boldsymbol{u}_i, \boldsymbol{u}_j \rangle$, the loss becomes:

$$\min_{\boldsymbol{x}} \ \mathcal{L}_S(\boldsymbol{x}) = -\frac{1}{|\mathcal{S}|} \sum_{s_{ij}^o \in \mathcal{S}} \log\left(1 + \exp\left(-\alpha s_{ij}^o s_{ij}^h\right)\right). \tag{E.1}$$

We use the W-type regularization:

$$h(\boldsymbol{Z}) = \sum_{i=1}^{n} h(\boldsymbol{z}_i), \quad h(\boldsymbol{z}) = \lambda \| \, |\boldsymbol{z}| - \boldsymbol{1} \, \|_1, \tag{E.2}$$

where $\lambda > 0$ controls regularization strength.

Combining (E.1) and (E.2) into (P) gives the final loss of DualHash in our experiments.

$$\begin{aligned} \min_{\boldsymbol{x} \in \mathcal{X}, \boldsymbol{B} \in \mathbb{R}^{n \times d}} \ & -\frac{1}{|\mathcal{S}|} \sum_{s_{ij}^o \in \mathcal{S}} \log\left(1 + \exp\left(-\alpha s_{ij}^o s_{ij}^h\right)\right) \\ & + \frac{\gamma}{2n} \sum_{i=1}^{n} \|\boldsymbol{b}_i - \boldsymbol{u}_i\|_2^2 + \sum_{i=1}^{n} \lambda \| \, |\boldsymbol{b}_i| - \boldsymbol{1} \, \|_1. \end{aligned}$$

Out-of-sample Extension. For an unseen query $\boldsymbol{a}_q \notin \boldsymbol{A}$, its binary code is predicted as:

$$\boldsymbol{b}_q = \text{sgn}(\boldsymbol{C}(\boldsymbol{x}; \boldsymbol{a}_q)).$$

Table E.1: Summary of symbols and notations.

Symbol	Description
$\boldsymbol{a}_i$, $\boldsymbol{y}_i$	Input feature and label vector for i-th sample
s_{ij}^o, s_{ij}^h	Similarity in input space and Hamming space
d_{ij}^o, d_{ij}^h	Distance in input space and Hamming space
$\boldsymbol{C}(\boldsymbol{x}; \boldsymbol{a}_i)$	Network output before activation
$\boldsymbol{b}_i \in \{-1, 1\}^d$	Binary hash code for i-th sample
$\boldsymbol{u}_i = \tanh(\boldsymbol{C}(\boldsymbol{x}; \boldsymbol{a}_i))$	Continuous code after activation
n, d	Number of samples and hash code length
$\boldsymbol{x}$	Network parameters
$d_{\boldsymbol{a}}$, d_x	Dimension of input feature and parameters

E.2. Implementation Details.

Baseline methods. We compare our method with eight baseline approaches, including one traditional discrete hashing method and seven deep supervised hashing methods:

(i) **SDH** [48]: A pointwise supervised hashing method using hand-crafted features. It employs discrete cyclic coordinate descent to directly optimize binary codes.

(ii) **DHN** [71]: A Bayesian framework that minimizes pairwise cross-entropy loss and quantization error using tanh relaxation and $\sum_i \log(\cosh(|z_i| - 1))$ regularization.

(iii) **DSH** [39]: Uses max-min pairwise loss with $\||\boldsymbol{z}| - \mathbf{1}\|_1$ regularization to minimize quantization error.

(iv) **DTSH** [57]: Learns hash codes by maximizing triplet likelihood with negative log-triplet loss, using $\||\boldsymbol{z}| - \mathbf{1}\|_2^2$ quantization regularization and a positive margin α to accelerate training and normalize Hamming distance gaps.

(v) **HashNet** [8]: Progressively increases β in $\tanh(\beta x)$ to approximate the sgn function, addressing nonconvex optimization challenges.

(vi) **DSDH** [36]: Integrates pairwise labels and classification information using alternating minimization to learn features and binary codes with the sgn function.

(vii) **OrthoHash** [28]: Uses a single cosine similarity loss to maximize alignment between continuous codes and binary orthogonal codes, ensuring discriminativeness with minimal quantization error.

(viii) **MDSHC** [55]: A two-stage method that first generates hash centers with minimal Hamming distance constraints, then uses three-part losses with $\||\boldsymbol{z}| - \mathbf{1}\|_1$ regularization to reduce quantization error.

Evaluation Protocols. Following [39], we adopt four evaluation metrics:

(i) *Mean Average Precision (mAP)*: For a query $\boldsymbol{x}_q$, the Average Precision is computed as:

$$\mathrm{AP}(\boldsymbol{x}_q) = \frac{\sum_{k=1}^{N} P(k) \cdot \mathbb{I}_{\mathrm{rel}}(k)}{\min(N, R_q)},$$

where $P(k)$ is precision at rank k, $\mathbb{I}_{\mathrm{rel}}(k)$ indicates whether the k-th result is relevant, R_q is the total relevant samples, and N is the number of retrieved items. mAP averages AP over all queries. For CIFAR-10, we use all returned neighbors; for NUS-WIDE, we consider top-5000 results.

(ii) *Precision at Top K (P@topK)*: The proportion of true neighbors among top-K retrieved results, averaged over all queries.

(iii) *Precision within Hamming Radius 2 (P@r=2)*: Measures retrieval accuracy for items within Hamming distance 2 from the query. This metric is particularly efficient as Hamming radius search has $\mathcal{O}(1)$ time complexity per query.

(iv) *Precision-recall curves of Hamming ranking*: PR curves illustrate precision at various levels of recall, offering insights into the trade-off between precision and recall at different thresholds. These curves are widely recognized as effective indicators of the robustness and overall performance of different algorithms.

Appendix F. Additional Experimental Results.

F.1. Comparison of Optimization Algorithms with Different W-Type Regularizations.

To validate the effectiveness of our primal-dual framework, we compare DualHash-StoM against optimization algorithms with different W-type regularizations. Table F.1 summarizes the configurations of different regularization functions and their corresponding optimization approaches. For StoMHash-WCR, we employ the regularization $\lambda\|\boldsymbol{z}^2 - \mathbf{1}\|_1$ following the iSPALM framework [45]. The proximal operator is computed with parameter $\rho > 0$ under the condition $\frac{\rho}{\eta} > \frac{1}{2}$, which ensures the subproblem is strongly convex with a unique solution.

Table F.1: Configurations of different W-type regularizations and optimization algorithms for DualHash-StoM and three baselines.

Method	$h(\boldsymbol{z})$	Convexity	Algorithm	Complexity
DSH [39]	$\|\|\boldsymbol{z}\| - \mathbf{1}\|_1$	Nonconvex	SGDM	–
StoMHash-WCR	$\|\boldsymbol{z}^2 - \mathbf{1}\|_1$	Weakly convex	SPGD	–
DHN [71]	$\sum_i \log(\cosh(\|z_i\| - 1))$	Nonconvex	SGDM	–
DualHash-StoM	$\|\|\boldsymbol{z}\| - \mathbf{1}\|_1$	Nonconvex	PD	✓

Notes: **Algorithm** describes the optimization strategy: SGDM (stochastic subgradient descent with momentum), SPGD (stochastic proximal gradient descent), PD (primal-dual). **Complexity** reports oracle complexity; '–' indicates no analysis available.

Training performance analysis. Figures F.1 and F.2 present training dynamics across different optimization approaches, comparing training loss, P@top1000, and P@r=2 metrics **for 64-bit codes** on CIFAR-10 and NUS-WIDE. On CIFAR-10, DualHash-StoM demonstrates strong optimization efficiency across all metrics. Our method achieves fast convergence in training loss while consistently maintaining high P@r=2 and P@top1000 values throughout

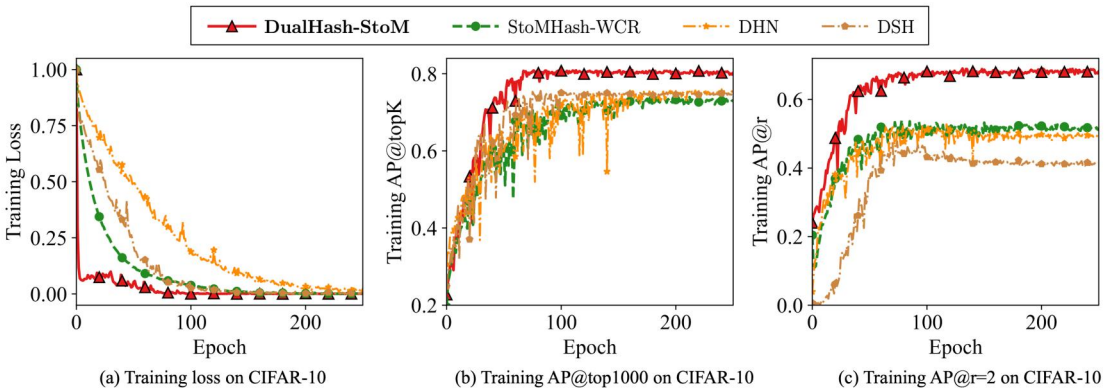


Figure F.1: Training performance comparison of DualHash-StoM and three baselines with 64-bit codes on the CIFAR-10 dataset.

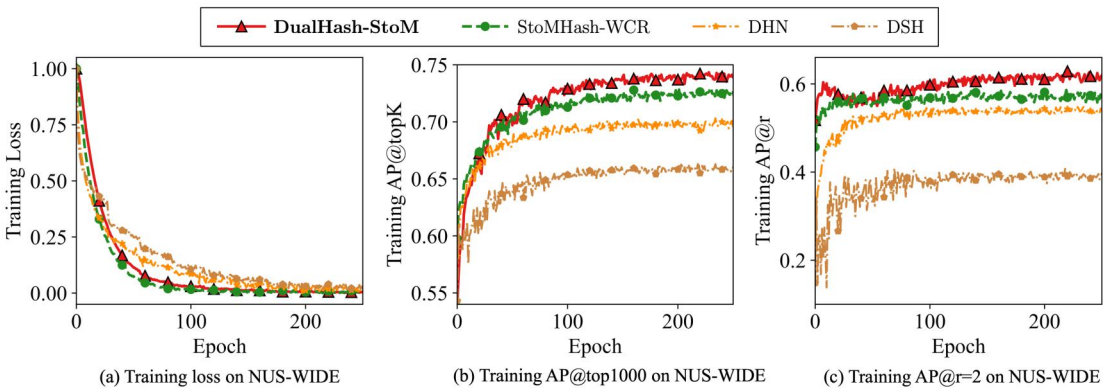


Figure F.2: Training performance comparison of DualHash-StoM and three baselines with 64-bit codes on the NUS-WIDE dataset.

the optimization process. While StoMHash with weakly convex regularization (StoMHash-WCR) shows competitive performance, DualHash-StoM achieves faster convergence and higher retrieval performance, demonstrating the effectiveness of our primal-dual transformation for handling W-type regularizations. On the more challenging NUS-WIDE dataset with complex multi-label scenarios, DualHash-StoM maintains competitive retrieval performance despite slightly slower initial convergence. The experimental results on diverse datasets indicate that utilizing nonconvex nonsmooth regularization with specialized primal-dual optimization can improve hash code learning and retrieval performance in practical applications.

Retrieval performance in Hamming space. We report mAP, P@r=2, and the average time per training epoch on CIFAR-10 and NUS-WIDE, all with 64-bit codes, for different methods in Table F.2. The results highlight the superior retrieval accuracy of DualHash-StoM compared to the baseline methods.

Specifically, on the CIFAR-10 dataset, DualHash-StoM achieves 9.9% and 27.2% im-

Table F.2: Retrieval performance comparison of DualHash-StoM and three baselines with 64-bit codes on CIFAR-10 and NUS-WIDE datasets.

Methods	CIFAR-10			NUS-WIDE		
	mAP@All	P@r=2	Time*	mAP@5000	P@r=2	Time*
DSH [39]	0.8010	0.4401	7.38	0.6634	0.4004	21.14
DHN [71]	0.8051	0.5272	**6.74**	0.7042	0.5432	**16.47**
StoMHash-WCR	0.7773	0.5402	7.04	0.7225	0.5857	16.69
DualHash-StoM	**0.8539**	**0.6872**	6.86	**0.7448**	**0.6548**	16.83

*Training time per epoch (seconds). **Bold** values indicate the best results.

provements over StoMHash-WCR in mAP@All and P@r=2, respectively. Similarly, on the NUS-WIDE dataset, DualHash-StoM outperforms StoMHash-WCR with a 3.1% improvement in mAP@5000 and an 11.8% improvement in P@r=2. Computationally, DualHash-StoM shows comparable efficiency to baselines such as DHN (e.g., 6.86s vs. 6.74s on CIFAR-10 and 16.83s vs. 16.47s on NUS-WIDE). This marginal time increase is justified by the substantial performance gains. Furthermore, Figure F.3 displays P@topK results and PR curves with 64-bit codes on CIFAR-10 and NUS-WIDE. DualHash-StoM consistently outperforms all comparison methods by large margins in P@topK. For PR curves, DualHash-StoM demonstrates superior performance at lower recall levels, making it especially well-suited for precision-critical image retrieval systems.

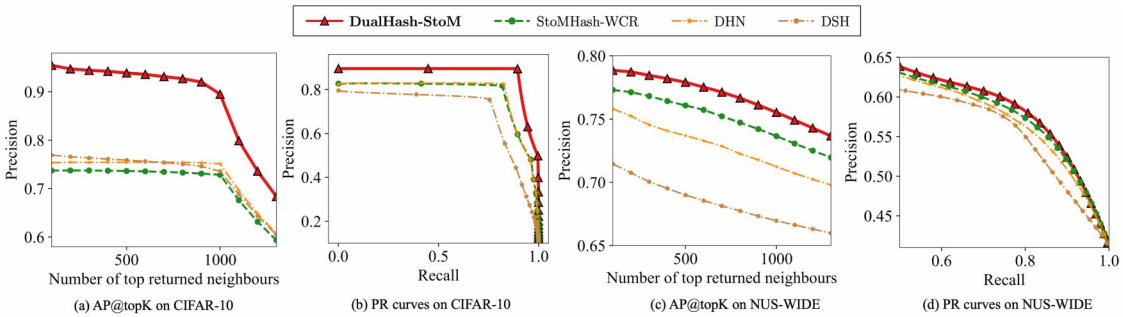


Figure F.3: Retrieval performance comparison of DualHash-StoM and three baselines with 64-bit codes on CIFAR-10 and NUS-WIDE datasets. (a), (c): P@topK results; (b), (d): PR curves.